



UNIVERSITY OF PISA
DEPARTMENT OF COMPUTER SCIENCE

Master programme in data science and business informatics

Data Mining: Advanced Topics and Applications Project report
IMDb Dataset

Professor:
Riccardo Guidotti

Group:
Truong Manh Nguyen
Thi Hien Do

Academic Year 2024/2025

Contents

1.Data Understanding and Preparation	1
1.1. Distribution Analysis.....	1
1.2. Missing Values Handling	1
1.3. Variable elimination and creation	2
1.4. Data Transformation	2
2. Outliers Detection	2
2.1. LOF	2
2.2. LODA	3
2.3. Isolation Forest.....	3
2.4. HBOS	3
2.5. CBLOF	3
2.6. ABOD	3
2.7. Visualization	3
3. Imbalance learning.....	5
3.1. Undersampling	6
3.2. Oversampling	7
3.3. Balancing at Algorithm level.....	7
3.4. Final discussion	8
4. Advanced Classification	8
4.1. Logistic Regression	9
4.2. Support Vector Machines	10
4.2.1. Linear SVM	10
4.2.2. Non-linear SVM	11
4.3. Neural Network.....	12
4.4. Ensemble Methods	13
4.4.1. Adaboost	13
4.4.2. Random Forest	14
4.4.3. Gradient Boosting Classifier	15
4.4.4. Extreme Gradient Boosting Classifier (XGBOOST)	16
4.5. Final Discussion	18
5. Advanced Regression	18
5.1. Neural Networks	18

5.2. Random Forrests	19
5.3. Gradient Boosting	20
5.4. Conclusion	21
6. Explainability	21
7. Time Series	23
7.1. Understanding and Preparation	23
7.2. Clustering.....	24
7.2.1. K-means.....	24
7.2.2. DBSCAN	25
7.3. Classification	26
7.3.1. KNN with Euclidean distance.....	26
7.3.2. KNN with DTW distance	26
7.3.3. Shapelets	27
7.3.4. Rocket	27
7.4. Motifs	28
7.5. Compare with Shapelets.....	29

1.Data Understanding and Preparation

The IMDb dataset expands significantly upon our initial analysis from the Data Mining 1 project. It now contains detailed information on 149,531 records and 32 features, of which 22 are numerical, 7 are categorical, and 3 are binary, capturing various aspects of movies and TV shows. The features are grouped as follows:

- **Numeric Features:** *startYear, endYear, runtimeMinutes, awardWins, numVotes, worstRating, bestRating, totalImages, totalVideos, totalCredits, criticReviewsTotal, awardNominationsExcludeWins, numRegions, userReviewsTotal, ratingCount, castNumber, companiesNumber, averageRating, externalLinks, writerCredits, directorsCredits, quotesTotal*
- **Categorical Features:** *originalTitle, rating, titleType, countryOfOrigin, genres, regions, soundMixes*
- **Binary Features:** *isAdult, canHaveEpisodes, isRatable*

1.1. Distribution Analysis

Most numeric features exhibit right-skewed distributions, with long tails extending toward higher values. These distributions indicate the presence of outliers and substantial variability across records—an expected characteristic of real-world media datasets, where a small number of highly popular titles typically dominate user engagement and visibility metrics.

Among the categorical features, *titleType* contains 10 distinct values with *tvEpisode* being the most frequent. *countryOfOrigin* and *regions* exhibit high cardinality, reflecting the dataset's broad geographical diversity, with *US* being the most common value in both. *soundMixes* also displays high cardinality with *mono* is being the most frequently used format, but a large proportion of entries are missing. The *genres* feature shows moderate variability, with *Comedy* as the most common category.

1.2. Missing Values Handling

The dataset contains missing values as table below:

	Missing Values	% missing
<i>endYear</i>	143,897	96.23
<i>runtimeMinutes</i>	40,195	26.88
<i>countryOfOrigin</i>	39,987	26.74
<i>regions</i>	34,824	23.28
<i>genres</i>	2,691	1.8
<i>soundMixes</i>	96,798	64.73

endYear: is relevant only for *tvSeries* and *tvMiniSeries*. Therefore, we removed it from further analysis.

runtimeMinutes: was imputed using the median runtime specific to each *titleType*.

countryOfOrigin, regions were imputed using the most frequent value in each column, which in both cases was "US". This approach reflects the dominant presence of U.S-produced content in the dataset.

genres were imputed using the mode of genres within each *titleType*, ensuring that missing values align with the typical thematic content of each category.

soundMixes was imputed by accounting for the historical evolution of audio technologies, using the most frequent sound format within each combination of *titleType* and production period. To support this strategy, we plotted the distribution of the five most common *soundMixes* over time (Fig.1.1) and observed clear technological progression—from silence in the early decades, to *mono* through the mid-20th century, followed by the rise of *stereo*, and more recently, the increasing adoption of *dolby_digital*.

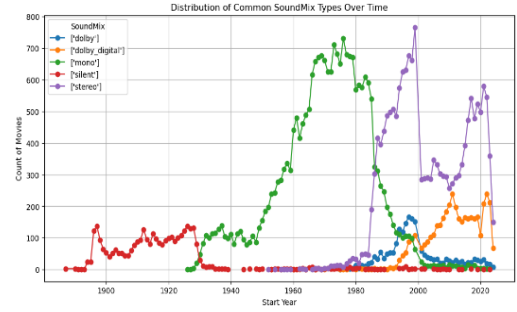


Figure 1.1: SoundMix distribution

1.3. Variable elimination and creation

To reduce redundancy and improve model efficiency, we analyzed the correlation matrix and identified variable pairs with correlation coefficients exceeding 0.8. The following pairs showed particularly high correlations: *numVotes* with *ratingCount* (1.00), *totalCredits* with *castNumber* (0.83), and *externalLinks* with *criticReviewsTotal* (0.92). Given these strong dependencies, we removed the redundant ones: *numVotes*, *externalLinks*, and *totalCredits*.

We also removed *worstRating* and *bestRating* as they contain constant values (1 and 10, respectively) across all records which provide no analytical insight. Similarly, *isRatable* was excluded due to its constant boolean value.

Additionally, to enhance the dataset's descriptive power, we created a new variable called *winningRate*, which captures the proportion of awards won relative to the total number of nominations and wins. This new metric enables fair comparisons across titles with varying levels of recognition, making it particularly useful for identifying relatively high-performing titles regardless of their nomination volume. Furthermore, we transformed the *countryOfOrigin* feature by aggregating individual countries into six continents—Asia, Europe, North America, South America, Africa, and Australia—resulting in a new variable called *continents*.

1.4. Data Transformation

To reduce complexity and prepare the dataset for modeling, high-cardinality categorical features were aggregated: *countryOfOrigin* into six continents, regions into the top 20 countries, and *soundMixes* into the five most common formats. All categorical variables then were frequency encoded, and features were standardized with a Standard Scaler to ensure comparability and reduce outlier influence.

2. Outliers Detection

In this chapter we explain how we identified the top 1% outliers in the tabular datasets, and how we deal with them. We used six different methods: LOF, LODA, ABOD, CBLOF, HBOS and Isolation Forest. In this study, we only used the dataset consisting of numerical attributes (18 attributes) and avoided using statistical approaches, as these methods assume the data follows a normal distribution, while our dataset exhibits a skewed distribution.

2.1. LOF

Since LOF is a density-based outlier detection method, its result is sensitive to the chosen hyperparameter values. After fixing the contamination parameter to 0.02, *n_neighbors* = 20. The results show 2512 outliers; these outliers are visualized via PCA in Fig. 2.1.

2.2. LODA

For LODA, we set parameters bins = 19 (this value is the optimize value that we find out based on Sturges method), n random cuts = 200, and contamination = 0.02. The result found 2991 outliers; these outliers are visualized via PCA in Fig. 2.1.

2.3. Isolation Forest

We applied Isolation Forest with default value, and it found 2991 outliers as 2% of the dataset (contamination = 0.02). The outliers are visualized via PCA in Fig. 2.1.

2.4. HBOS

For HBOS, we set parameters bins = 19 (this value is the optimize value that we find out based on Sturges method), alpha = 0.1 to prevent bins with zero density by adding a small value to each bin, tol = 0.5 to adjust the sensitivity of the algorithm towards data points near the edges of histogram bins and contamination = 0.02. The result found 2991 outliers; these outliers are visualized via PCA in Fig. 2.1.

2.5. CBLOF

For CBLOF, we set parameters alpha = 0.9 (almost equal to 1) for easier to find out outliers; contamination = 0.02; n_clusters = 3 to enhance the difference between small and big clusters. The result found 2991 outliers; these outliers are visualized via PCA in Fig. 2.1.

2.6. ABOD

We applied contamination = 0.02 and method = 'fast' to consider only the nearest points instead of the entire dataset, which improves computation speed. In addition, we set n_neighbors = 20 after experimenting with values starting from 10. The algorithm was only able to run with n_neighbors set to 17 or higher. We chose 20 to ensure accurate outlier detection while maintaining computational efficiency. The result identified 2,959 outliers, which are visualized using PCA in Fig. 2.1.

2.7. Visualization

First, we applied PCA to visualize the outliers and inliers detected by each method, as shown in Fig. 2.1. In general, all methods indicate that outliers lie far from the central clusters. However, each method has its own way of defining outliers, resulting in slightly different visualizations.

With HBOS, most outliers (red points) are clearly separated from the dense central region of inliers (blue points). However, some inliers appear mixed within the outliers. We attribute this issue to HBOS's limitation of treating attributes independently, thereby missing multi-dimensional correlations, leading to misclassification of complex structured outliers.

For ISO, the inliers predominantly distribute along a vertical axis and are more clearly separated compared to HBOS. However, ISO's inherent randomness and ineffectiveness in distinguishing closely located outliers and inliers are visible in the visualization. Specifically, two inliers appear notably distant from the main group of inliers, while one outlier appears unusually close to the inlier cluster compared to other outliers.

The LODA method yields significantly different results in classifying outliers and inliers compared to ISO and HBOS. However, LODA suffers from inconsistency due to its randomness, resulting in different outcomes upon repeated executions.

ABOD and CBLOF employ two distinct approaches—angle-based measurements among vectors for ABOD and distance-based clustering for CBLOF. Yet, despite these methodological differences, their PCA-based visualizations reveal quite

similar results. Moreover, these two methods provide the clearest distinction between outliers and inliers when compared to the remaining methods.

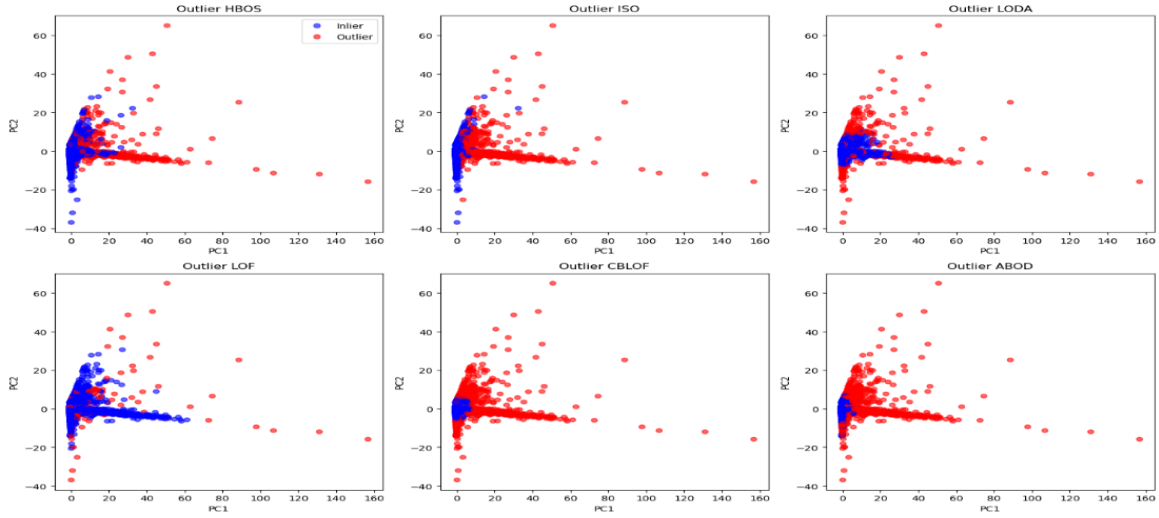


Figure 2.1: PCA Visualization for Outliers in each method

Among them, LOF stands out due to its unique approach of computing the local neighborhood density for each data point. As a result, PCA highlights the most noticeable differences between LOF and other methods.

Additionally, since PCA is a linear technique, reducing the data to only two dimensions fails to capture the true structure of the dataset. This leads to misleading impressions—for example, some plots from ISO, ABOD, and CBLOF appear to show more outliers than inliers, even though the contamination rate was only 0.02, meaning the actual proportion of outliers is at most 2% of the entire dataset. Therefore, we decided to plot a Venn diagram to illustrate the similarities in the outlier results of several methods in Fig. 2.2.

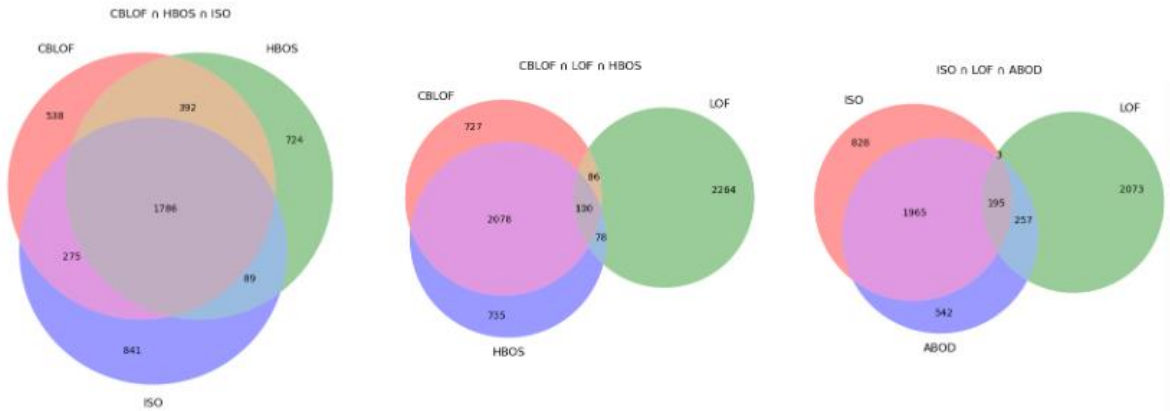


Figure 2.2: Venn graph for comparison

It can be seen that although the PCA plots for HBOS and CBLOF look very different, the proportion of outliers they identify overlaps by as much as about 75%. A similar pattern emerges when comparing the outlier results of ISO and ABOD. If one were to rely solely on PCA visualization, one would perceive a large difference in the outliers and inliers detected by these two methods. However, based on the Venn diagram, these two methods share approximately 73% of their outliers. In contrast, LOF continues to demonstrate its distinct local approach to outlier detection—its overlap with other methods such as CBLOF, HBOS, ABOD, and ISO remain below 18%.

From the analyses above, we conclude that each method brings unique insights based on its underlying assumptions. Therefore, to select a consensus set of outliers and visualize them more clearly, we applied a majority voting strategy—specifically, a record is considered an outlier if it is identified as such in at least 4 out of 6 methods. The result yielded 1,533 outliers satisfying this condition (approximately 1% of the dataset). We then visualized these results using both PCA and t-SNE, as shown in Fig. 2.3.

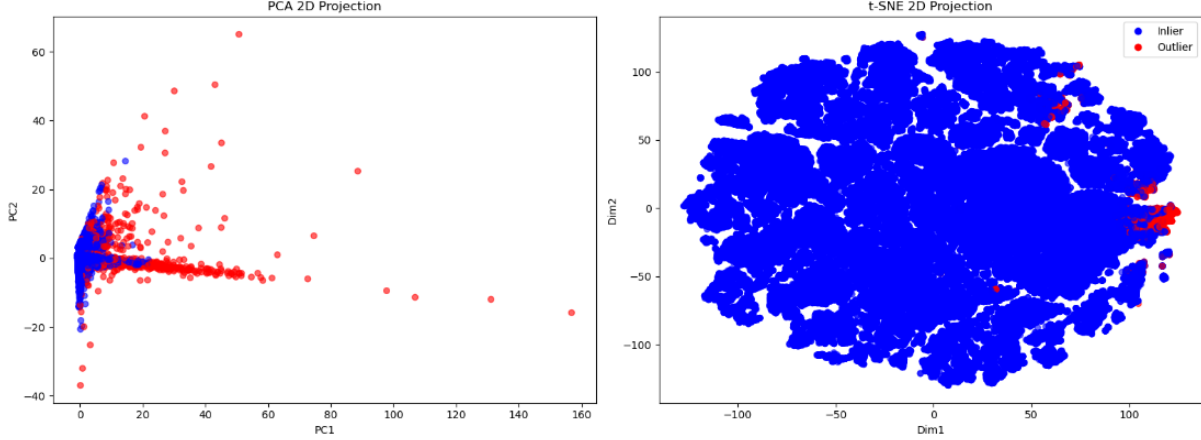


Figure 2.3: PCA and t-SNE Visualization for Outliers

It can be observed that PCA still fails to clearly visualize the outliers. In contrast, t-SNE shows the opposite by capturing non-linear relationships and preserving the distances between nearby points in the original space. t-SNE also forms very distinct clusters, where the outliers are grouped into three clearly separated clusters. This result is consistent with the outlier clustering outcome using K-Means.

Finally, we decided to remove all of outliers out of the dataset. By removing these records, we mitigate the risk of false positives and negatives, which could otherwise skew the results of our analysis.

3. Imbalance learning

In this session, we addressed the challenge of imbalanced classification by defining a simple binary classification task using rating attribute.

We selected two categories: (7, 8] as the majority class or negative class and (9, 10] as the minority class or positive class, discarding all other classes to isolate a two-class problem. In the training set, class (7, 8] consisted of 33,617 instances, while class (9, 10] consisted of only 2,990 instances, resulting in an imbalance ratio of approximately 91.8% vs. 8.2% (Fig. 3.1). As this distribution naturally reflected a real-world imbalance where very few titles receive exceptionally high ratings, which aligned well with objectives of our task. Therefore, we retained the original imbalance without any synthetic manipulation.

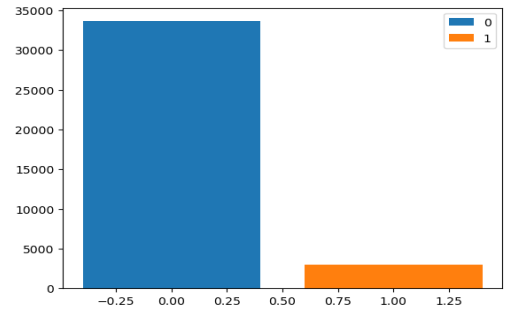


Figure 3.1: Distribution of classes

We selected a Decision Tree classifier trained on the original data without any balancing technique as our baseline model to evaluate classification performance and compare it against various imbalance learning methods. We performed hyperparameter tuning by Randomized Search and selected the best configuration as: `'min_samples_split': 5`, `'min_samples_leaf': 1`, `'max_depth': None`, `'criterion': 'gini'`. Although the model achieved high accuracy 0.9, the performance on the minority class was poor, with a precision of 0.38, recall of 0.37, and F1 score of 0.37. This highlights

the classic misleading of accuracy in imbalanced settings, where a model can be accurate overall but fails to detect rare but important cases. The ROC AUC and Precision- Recall AP were 0.68 and 0.22 respectively, indicating weak minority class separability.

To address the imbalance, we implemented three main techniques: Undersampling, Oversampling, and Balancing at the algorithm level. For all methods, we tuned the hyperparameters using Randomized Search after resampling to optimize performance; however, these tuning details are not reported here due to space limitations.

Below, Table 3.1 and Figure 3.4 show the detailed performance metrics of the minority class for all the methods applied in our analysis:

Table 3.1: Result comparison

Methods	Class Distribution	Precision (Class 1)	Recall (Class 1)	F1 (Class 1)	Accuracy
Decision Tree Baseline	{0: 33617, 1: 2990}	0.38	0.37	0.37	0.90
Random Under Sampler	{0: 2990, 1: 2990}	0.16	0.80	0.27	0.64
Condensed NN	{0: 5272, 1: 2990}	0.19	0.48	0.27	0.79
Edited NN	{0: 29004, 1: 2990}	0.32	0.44	0.37	0.88
Tomek Links	{0: 32778, 1: 2990}	0.35	0.41	0.38	0.89
Cluster Centroids	{0: 2990, 1: 2990}	0.13	0.88	0.22	0.50
Random Over Sampler	{0: 33617, 1: 33617}	0.37	0.36	0.36	0.90
SMOTE	{0: 33617, 1: 33617}	0.24	0.67	0.35	0.80
ADASYN	{0: 33617, 1: 33524}	0.23	0.61	0.34	0.81
Class Weight	{0: 33617, 1: 2990}	0.35	0.35	0.35	0.89
Decision Threshold	{0: 33617, 1: 2990}	0.32	0.44	0.37	0.88

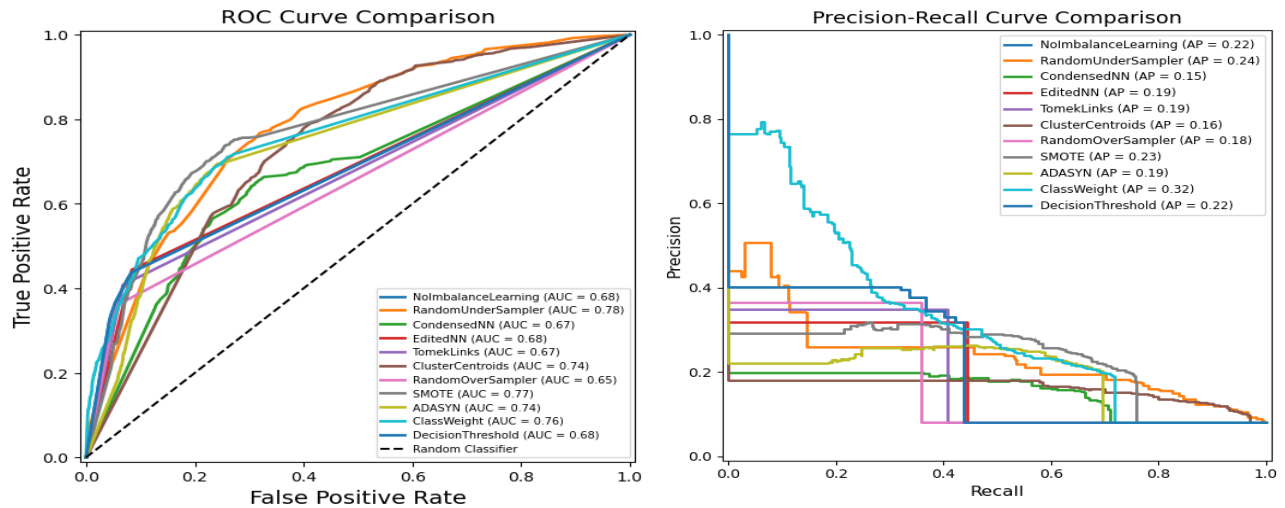


Figure 3.4: ROC Curves and Precision-Recall Curves

3.1. Undersampling

Undersampling methods attempt to rebalance the dataset by reducing the number of majority class instances.

Random Under Sampler: randomly reducing the majority class to balance the dataset but it also discards a significant amount of information. It improved recall significantly to 0.80 and had the highest ROC AUC of 0.78 among all methods. However, it came at the cost of a sharp drop in precision 0.16 and overall accuracy 0.64, due to increased false positives.

Condensed Nearest Neighbors (CNN) heavily reduced the majority class to 5272 instances, creating a partially balanced dataset. The model achieved a moderate recall of 0.48 and precision of 0.19, resulting in an F1 score of 0.27. Although it improved recall over the baseline, the low precision and AP of 0.15, combined with an ROC AUC of 0.67, indicate limited effectiveness.

Edited Nearest Neighbors (ENN) reduced the majority class by eliminating instances that conflicted with their neighbors. The dataset remained heavily imbalanced with 29,004 majority class instances. This method produced a slightly lower recall of 0.44 than CNN, with precision at 0.32, resulting in an F1 score of 0.37. It maintained higher accuracy 0.88 by retaining more majority samples than CNN. However, its ROC AUC 0.68 and AP 0.19 remained moderate, indicating modest improvement over the baseline.

Tomek Links removed borderline majority class instances to clarify decision boundaries, leaving 32,778 majority and 2,990 minority samples while preserving most of the original data. This method offered the most balanced performance among Undersampling methods, with precision 0.35, recall 0.41, and an F1 score of 0.38 slightly higher than the baseline.

Cluster Centroids aggressively reshaped the majority class by replacing it with 2,990 cluster centroids, aligning it with the size of the minority class. It achieved the highest recall of 0.88, but with the cost is extremely low precision of 0.13 and a very poor accuracy of 0.50, the AP was only 0.16. These results suggest that while Cluster Centroids is effective for maximizing sensitivity in minority class, it over-simplifies the majority class distribution leading to excessive false positives.

3.2. Oversampling

Oversampling methods increase the representation of the minority class.

Random Over Sampler simply replicates existing minority class samples to balance the class. This naïve method did not significantly alter the model's behavior: recall remained at 0.36, precision at 0.37. Both the ROC curve and Precision-Recall curve closely resembled those of the baseline model indicated little to no real improvement.

SMOTE generated perfectly balanced datasets with 33,617 samples in each class. It achieved the best performance between oversampling methods, with a recall of 0.67, ROC AUC of 0.77, and AP of 0.23. By generating synthetic minority samples, SMOTE improves the model's generalization toward the minority class.

ADASYN also produced a nearly balanced dataset with 33,617 majority and 33,524 minority samples achieved a recall of 0.61, slightly lower than SMOTE, with ROC AUC of 0.74 and AP of 0.19. Unlike SMOTE, ADASYN focuses on harder-to-learn samples by generating more synthetic data near the decision boundary. While this helps the model focus on challenging regions of the feature space, it may also introduce noise. In this case, it was slightly less effective than SMOTE.

3.3. Balancing at Algorithm level

These techniques adjust the learning algorithm, thereby preserving the original class distribution, and aim to address the unfairness caused by skewed class distributions that can bias the model toward the majority class.

Class Weight Adjustment: The Decision Tree classifier was fine-tuned using the following hyperparameter configuration: `'min_samples_split': 30, 'min_samples_leaf': 5, 'max_depth': 19, 'criterion': 'entropy', 'class_weight': {0: 1, 1: 2}}`. This configuration assigns twice the penalty to misclassified instances of the minority class. This method provided stable and effective performance across all evaluation metrics, achieving the highest AP 0.32 among all methods, strong ROC AUC 0.76, and balanced precision 0.35 and recall 0.35.

The Decision Threshold Moving is a post training method that was applied by adjusting the classification threshold of the baseline Decision Tree model. Instead of using the default threshold of 0.5, we systematically tested different thresholds ranging from 0.1 to 0.9. The goal was to find the threshold that would maximize the recall for the minority class while also improving the F1-score, which balances precision and recall. After evaluating the performance at each threshold, we selected 0.1 as the optimal threshold. At this setting, it improved recall to 0.44, higher than the baseline 0.37, while maintaining reasonable precision 0.32. However, its ROC AUC and AP remained the same as the baseline. The adjustment only shifted the trade-off between precision and recall; it did not enhance the model’s overall discriminatory ability.

3.4. Final discussion

Our analysis demonstrates that handling class imbalance is crucial for effective minority class detection. While Undersampling methods like Cluster Centroids and Random Under Sampler maximize recall, they severely compromise precision and accuracy as discarding large amounts of information from the majority class leads to a less representative dataset and poorer generalization. Oversampling methods like SMOTE provide a balanced improvement with good recall and acceptable precision. Balancing at the algorithm level, particularly adjusting Class Weight emerged as a robust technique, offering the best overall trade-off between recall, precision, and stability.

To our opinions, the choice of the "best" method ultimately depends on the context and the specific goals of the task. If the purpose is detecting as many minority instances as possible is prioritized, Cluster Centroids and Random Under Sampler would be the preferred approaches. However, if the goal is to build a film recommendation system for users where a balance between false positives and true positives is critical, SMOTE would likely be the best choice due to its stronger generalization ability. Alternatively, if a fast and lightweight solution is needed without any manipulation of the dataset, Class Weight adjusting provides quite good results while preserving the original data structure.

4. Advanced Classification

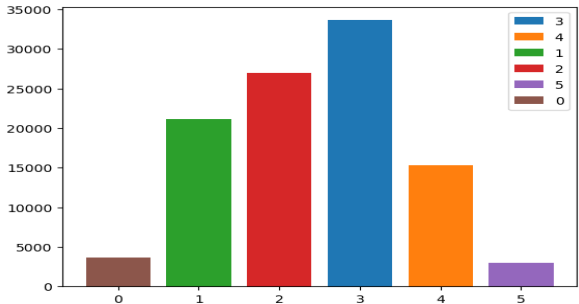
In this task, we aim to solve a multi-class classification problem by predicting the *rating* variable using advanced machine learning methods: Logistic Regression, Support Vector Machines (SVM), Neural Networks, Ensemble Methods (Random Forest, Adaboost, Gradient Boosting Machines (GBM), Extreme Gradient Boosting Machines (XGBoost)).

Originally, the dataset contained ratings divided into 10 distinct intervals. To reduce data sparsity in underrepresented groups and to facilitate more effective classification, these intervals were consolidated into 6 broader classes as follows:

Table 4.1: Rating Class Intervals and Interpretations

Class	Rating Interval	Interpretation
0	(0, 4]	Very Low
1	(4, 6]	Low
2	(6, 7]	Moderate
3	(7, 8]	Good
4	(8, 9]	Very Good
5	(9, 10]	Excellent

Figure 4.5: Distribution of Class Instances



The resulting class distribution remains highly imbalanced (Fig. 4.5) with Class 3 notably contains the most instances, whereas class 0 and class 5 are underrepresented. This imbalance poses a serious challenge for classification, as models tend to be biased toward the majority classes, often leading to reduced performance in predicting the minority classes.

Additionally, exploratory data visualizations using dimensionality reduction techniques such as PCA (Principal Component Analysis) and UMAP (Uniform Manifold Approximation and Projection) (Fig. 4.2) reveal substantial overlap among classes. While PCA captures only linear patterns, UMAP preserves non-linear relationships, yet both techniques demonstrate limited class separability, making our classification tasks more complicated.

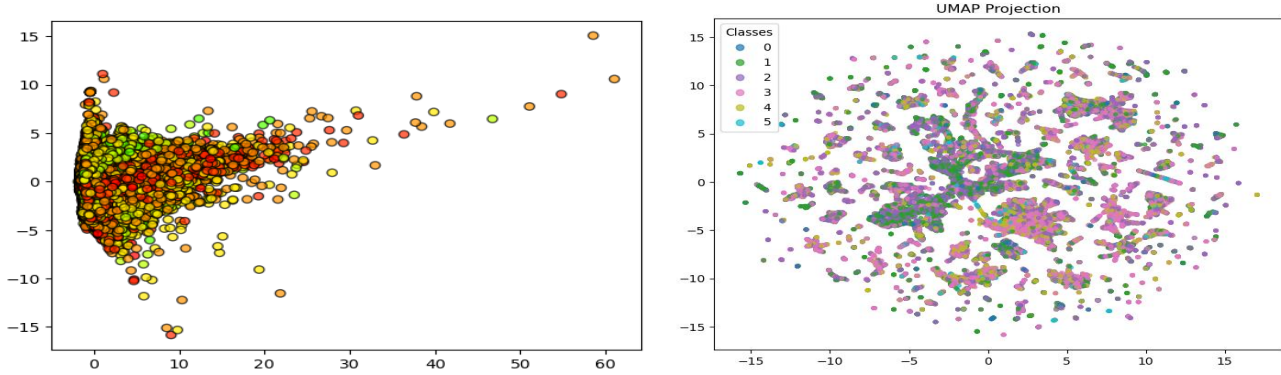


Figure 4.2: PCA and UMAP Visualizations

Given these challenges, we evaluated several imbalances handling techniques, including SMOTE, SMOTE-Tomek, class weight adjustment, and decision threshold moving. However, in our experiments, only algorithm-level balancing techniques consistently performed well across different classifiers.

As a result, we adopted class weight adjustment for models that support it, specifically Logistic Regression, SVM and Ensemble Methods. For the Neural Network, we employed decision threshold moving, which enabled us to fine-tune the prediction probabilities to better account for class imbalance without modifying the model architecture.

4.1. Logistic Regression

The grid search systematically evaluated various combinations of the following hyperparameters to identify the optimal configuration for the logistic regression model:

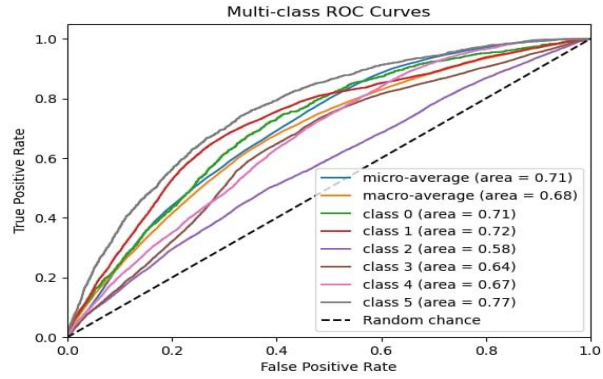
- Regularization Strength (C): The search space included values of [0.001, 0.01, 0.1, 1, 10, 100] to control the trade-off between fitting the training data well and keeping the model simple to avoid overfitting.
- Penalty Norm (penalty): Both L1 and L2 penalties were considered to explore different approaches to regularizing model complexity.
- Optimization Algorithm (solver): A set of solvers ['lbfgs', 'liblinear', 'newton-cholesky', 'sag', 'saga'] was tested to find the most effective algorithm for converging to the optimal model weights.

The optimal hyperparameter combination was determined to be a *regularization strength (C) of 0.1, an L1 penalty, and the 'liblinear' solver*. A comprehensive summary of the cross-validation results for all evaluated combinations and ROC Curve are detailed in Table 4.2 and Fig. 4.3 respectively.

Table 4.2: Logistic Regression result

Class	Precision	Recall	F1-score
0	0.10	0.20	0.13
1	0.40	0.55	0.47
2	0.36	0.09	0.14
3	0.46	0.55	0.50
4	0.24	0.07	0.10
5	0.09	0.46	0.15
Accuracy			0.35
Macro avg	0.27	0.32	0.25
Weighted avg	0.37	0.35	0.32

Figure 4.3: Logistics Regression ROC Curve



The Logistic Regression classifier achieved 35% accuracy, a macro-average F1-score of 0.25, and a macro-average AUC of 0.68, indicating weak overall performance. It performed best in Classes 1 and 3 (F1 = 0.47 and 0.50), although Class 3, despite being a well-represented class, had low AUC (0.64), suggesting poor separability despite a decent F1 score. Class 2, another majority class, was also poorly classified (F1 = 0.14, AUC = 0.58), showing the model's difficulty in handling even dominant categories. Class 4 also showed a very low recall (0.07) and F1-score (0.10) despite a moderate AUC of 0.67, indicating that while the model could somewhat separate the class, it failed to identify most of its instances. For minority Classes 0 and 5, the model achieved only modest recall (0.20 and 0.46) and very low precision (0.10 and 0.09), resulting in low F1-scores and a high number of false positives. Overall, Logistic Regression struggled with both class imbalance and overlapping, failed to generalize well across classes.

4.2. Support Vector Machines

4.2.1. Linear SVM

Hyperparameter tuning was conducted for the Linear SVM to identify the optimal model configuration. The search grid included regularization strength (C) values of [0.001, 0.01, 0.1, 1, 10, 100] and both L1 and L2 for the penalty norm. The dual parameter was set to False because the L1 penalty does not support the dual formulation.

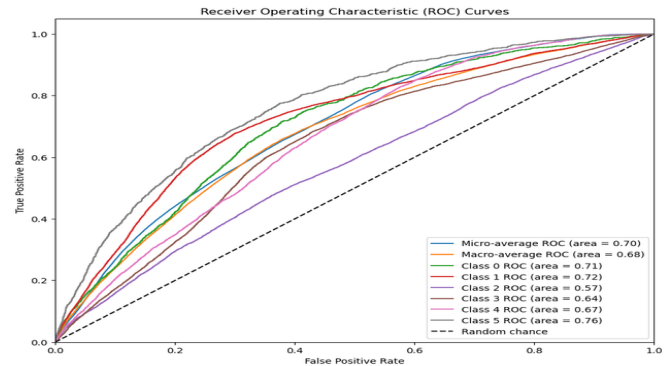
Based on the cross-validation results, the optimal hyperparameter combination was a C value of 0.01 with an L1 penalty.

Table 4.3 and ROC Curve (Fig. 4.4) provide a detailed summary of the performance for these hyperparameter combinations.

Table 4.3: Linear SVM result

Class	Precision	Recall	F1-score
0	0.10	0.16	0.12
1	0.39	0.58	0.47
2	0.37	0.07	0.12
3	0.46	0.57	0.51
4	0.22	0.05	0.09
5	0.09	0.46	0.15
Accuracy			0.35
Macro avg	0.27	0.31	0.24
Weighted avg	0.36	0.35	0.31

Figure 4.4: Linear SVM ROC Curve



The Linear SVM classifier achieved 35% accuracy, a macro-average F1-score of 0.24, and a macro-average AUC of 0.68, performing similarly to Logistic Regression. Therefore, the performance across classes can be interpreted in the same way. Overall, it faced similar issues with class imbalance and poor separability.

4.2.2. Non-linear SVM

Due to the significant computational cost of non-linear SVMs, which stems from the calculation of the kernel matrix for all data point pairs (with a complexity of $O(n^2)$ to $O(n^3)$) and the necessity of training multiple models for the One-vs-One multi-class strategy, a modified approach was employed. We partitioned the dataset into ten subsets and randomly selected five of them to perform a grid search for hyperparameter optimization. The process was conducted separately for each kernel—'rbf', 'sigmoid', and 'poly'—by tuning the regularization parameter C and the gamma parameter, which controls the influence radius of a single training example.

The optimal hyperparameter sets identified from the five tuning runs were:

- {'C': 100, 'gamma': 0.001, 'kernel': 'rbf'}
- {'C': 10, 'gamma': 'scale', 'kernel': 'sigmoid'}
- {'C': 1, 'gamma': 'auto', 'kernel': 'poly'}

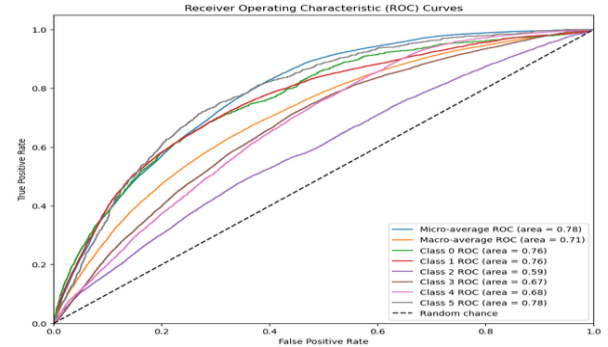
Subsequently, these three parameter configurations were evaluated on the full dataset. The model with a polynomial kernel ({'C': 1, 'gamma': 'auto', 'kernel': 'poly'}) emerged as the best-performing non-linear model, achieving an accuracy of approximately 0.28 and macro average for precision and recall as 0.29 and 0.35 respectively. This result significantly outperformed the rbf and Sigmoid configurations, which yielded accuracies of 0.27 and 0.16, respectively.

Detailed performance metrics for this optimal non-linear model are presented in Table 4.4 and Fig. 4.5.

Table 4.4: Non-linear SVM result

Class	Precision	Recall	F1-score
0	0.10	0.45	0.16
1	0.44	0.40	0.42
2	0.42	0.09	0.14
3	0.49	0.36	0.42
4	0.23	0.17	0.20
5	0.07	0.64	0.13
Accuracy			0.28
Macro avg	0.29	0.35	0.24
Weighted avg	0.40	0.28	0.30

Figure 4.5: Non-linear SVM ROC Curve



The non-linear SVM classifier achieved 28% accuracy, a macro-average F1-score of 0.24, and a macro-average AUC of 0.71, performing slightly worse than Linear SVM in terms of accuracy but similarly in F1-score. Like the linear model, it performed moderately on Classes 1 and 3 but continued to struggle with Class 2 (F1 = 0.14, AUC = 0.59) and Class 4 (F1 = 0.20, AUC = 0.68). It showed slightly improved recall on minority classes such as Class 0 (0.45) and Class 5 (0.64), along with better ROC-AUCs, but their very low precision led to weak F1-scores. Overall, the non-linear model offered no clear advantage over the linear one, continued to suffer from class imbalance and overlapping distributions, and was computationally much more expensive.

4.3. Neural Network

We evaluated three different neural network configurations:

- Two hidden layers with 32 and 16 units
- Three hidden layers with 64, 32, and 16 units
- Three hidden layers with 128, 64, and 32 units

All models used a SoftMax output layer with 6 units, corresponding to the 6 target classes.

To optimize our models, we used Keras Tuner with the RandomSearch strategy. The objective was to maximize validation accuracy. To mitigate overfitting and improve generalization, we incorporated the following techniques:

- Dropout regularization randomly disconnects a fraction of neurons within layer during training.
- EarlyStopping was used with val_accuracy as the monitor metric and a patience of 20 epochs. Training stopped early when no improvement was seen.

The following hyperparameters were tuned:

- Activation function: relu or tanh
- Dropout rate: ranged from 0.1 to 0.5 with step size = 0.1
- Learning rate: [0.001, 0.01, 0.1]
- Optimizer: ADAM

We used the Adam optimizer during training because Adam combines the benefits of momentum and RMSprop to adjust the learning rate for each parameter automatically, leading to faster and more stable convergence. We allowed 150 trials with 1 execution per trial. The best models were selected based on validation accuracy and validation loss. The tables below show the best hyperparameters and corresponding performance for each model:

Table 4.5: Best hyperparameters result

Hidden layer	(32, 16)	(64, 32, 16)	(128, 64,32)
Activation_1	Tanh	ReLu	ReLu
DropOut_1	0.2	0.2	0.2
Activation_2	ReLu	ReLu	ReLu
DropOut_2	0.2	0.2	0.2
Activation_3		Tanh	ReLu
DropOut_3		0.3	0.3
Optimizer	ADAM	ADAM	ADAM
Learning rate	0.001	0.001	0.001
Validation Accuracy	0.42	0.43	0.44
Validation Loss	1.35	1.33	1.30

The best performing model was the one with 3 hidden layers (128, 64, 32), achieving the highest validation accuracy and lowest the validation loss among all test configurations. The learning curves showed that training and validation accuracy

converged smoothly, and no significant overfitting was observed thanks to the regularization techniques. Training stopped at Epoch 121 due to EarlyStopping, triggered by no further improvement in validation accuracy.

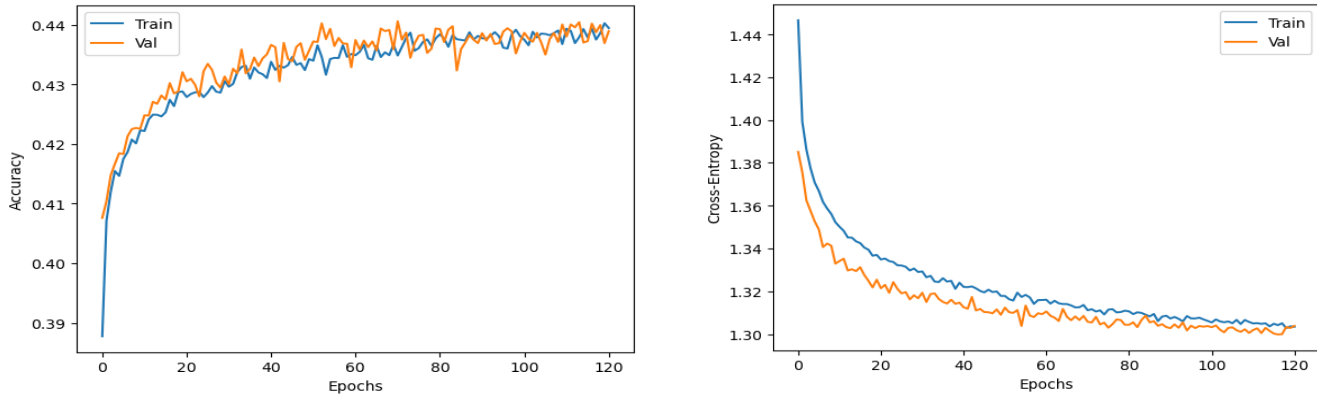


Figure 4.6: Learning curves of the neural network

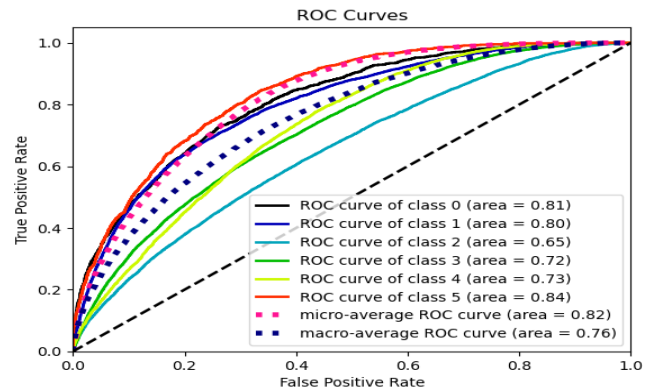
To further improve evaluation, we applied threshold tuning using a random search-based strategy to optimize the macro F1-score. The best thresholds were found to be [0.11, 0.74, 0.82, 0.64, 0.23, 0.45], resulting in a best macro F1-score of 0.32 on the test set.

The final performance metrics after threshold optimization are summarized below:

Table 4.6: Neural network result

Class	Precision	Recall	F1-score
0	0.30	0.17	0.22
1	0.51	0.45	0.48
2	0.39	0.28	0.33
3	0.42	0.53	0.46
4	0.29	0.39	0.33
5	0.74	0.05	0.10
Accuracy			0.40
Macro avg	0.44	0.31	0.32
Weighted avg	0.42	0.40	0.39

Figure 4.7: Neural network ROC Curves



The neural network model achieved an accuracy of 40%, a macro-average F1-score of 0.32, and a macro-average AUC of 0.76, showing improved overall performance compared to 3 models above. It performed well in Classes 1 and 3 (F1 = 0.48 and 0.46) and showed strong class separability, particularly for Classes 0 and 5 (AUC = 0.81 and 0.84). Notably, Classes 2 and 4 showed a much-improved F1-score (0.33 each) compared to previous models, where they consistently underperformed. Class 0 and Class 5 still suffered from very low recall, resulting in a weak F1-score. Overall, the neural network provided better class separability and performance, especially in previously difficult classes but struggled with recall on minority classes.

4.4. Ensemble Methods

4.4.1. Adaboost

AdaBoost combines multiple weak learners typically decision stumps with only one node and two leaves to form a strong classifier. It iteratively adjusts the weights of training samples, emphasizing those misclassified by previous learners. We applied hyperparameter tuning to optimize the performance of the AdaBoost classifier. A randomized search was carried out over the following parameter grid:

'n_estimators': [20, 50, 100, 200, 300, 500],

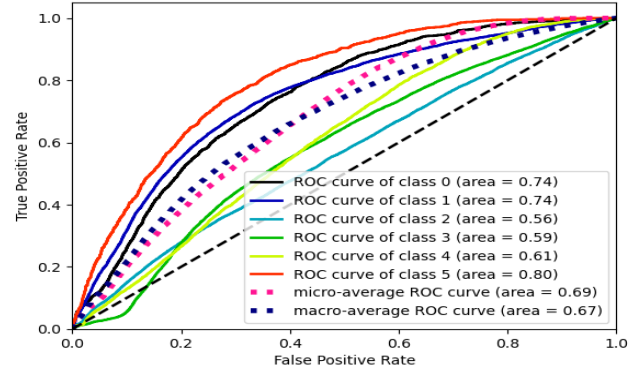
'learning_rate': [1.0, 0.5, 0.2, 0.1, 0.05, 0.01]

The best hyperparameters identified were $n_estimators = 500$, $learning_rate = 0.5$

Table 4.7: AdaBoost result

Class	Precision	Recall	F1-score
0	0.10	0.45	0.17
1	0.45	0.39	0.42
2	0.38	0.14	0.21
3	0.52	0.38	0.44
4	0.24	0.21	0.22
5	0.09	0.63	0.15
Accuracy			0.30
Macro avg	0.30	0.37	0.27
Weighted avg	0.40	0.30	0.32

Figure 4.8: Neural Network ROC Curve
ROC Curves



The AdaBoost classifier achieved an accuracy of 30%, a macro-average F1-score of 0.27, and a macro-average AUC of 0.67, reflecting limited overall performance. The model performed best on Class 1 and Class 3 (F1-scores of 0.42 and 0.44, respectively), while Class 2, despite its large support, had a weak performance (F1 = 0.21, AUC = 0.56), indicating poor separability. Class 4 also performed poorly (F1 = 0.22, AUC = 0.61). Classes 0 and 5 had relatively high recall (0.45 and 0.63) but very low precision, leading to weak F1-scores and many false positives. ROC curves show that only Class 5 reached a strong AUC (0.80), while others remained moderate or low. Due to the use of decision stumps as base learners, the model lacked the capacity to capture complex relationships in the data. As a result, AdaBoost struggled to generalize well across most classes.

4.4.2. Random Forest

We performed hyperparameter tuning using grid search across a broad parameter space. We tuned both the number of trees and the structure of each individual tree to improve generalization and reduce overfitting. Specifically, we tested the following values:

- $n_estimators$: [50, 100, 200, 300, 400, 500]
- max_depth : [9, 10, 11, 12]
- $min_samples_split$: [2, 5, 10, 20]
- $min_samples_leaf$: [1, 5, 10, 20]
- $max_features$: ['sqrt', 'log2']
- $criterion$: ['gini', 'entropy']

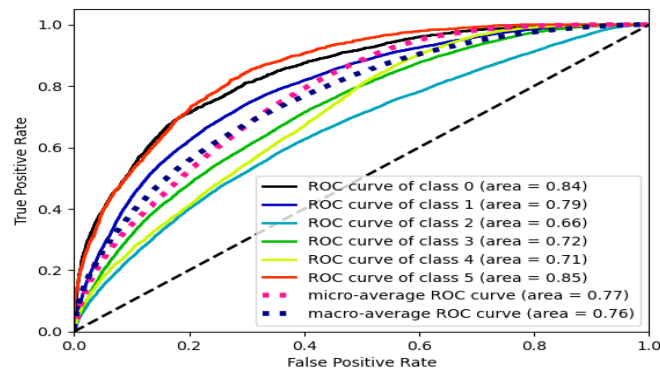
The best parameter configuration found was:

`{'n_estimators': 400, 'min_samples_split': 2, 'min_samples_leaf': 1, 'max_features': 'sqrt', 'max_depth': 1, 'criterion': 'gini'}`

Table 4.8: Random Forest result

Class	Precision	Recall	F1-score
0	0.18	0.47	0.26
1	0.45	0.50	0.48
2	0.45	0.21	0.28
3	0.55	0.42	0.48
4	0.30	0.29	0.29
5	0.11	0.62	0.18
Accuracy			0.37
Macro avg	0.34	0.42	0.33
Weighted avg	0.44	0.37	0.38

Figure 4.9: Random Forest ROC Curve



The Random Forest classifier achieved an accuracy of 37%, a macro-average F1-score of 0.33, and a macro-average AUC of 0.76, showing an improvement over AdaBoost. It performed well in Classes 1 and 3 (F1 = 0.48 each), while Classes 2 and 4 showed improvements (F1 = 0.28 and 0.29; AUC = 0.66 and 0.71). Minority Classes 0 and 5 also improved, with Class 0 achieving F1 = 0.26 (AUC = 0.84), and Class 5 reaching high recall (0.62) but low precision, leading to an F1-score of 0.18. The ROC curves confirmed good separability for Class 0 and Class 5 (AUCs of 0.84 and 0.85), whereas Class 2 showed the weakest discrimination (AUC = 0.66). Overall, Random Forest provided better class separability and generalization than AdaBoost, especially for difficult and underrepresented classes.

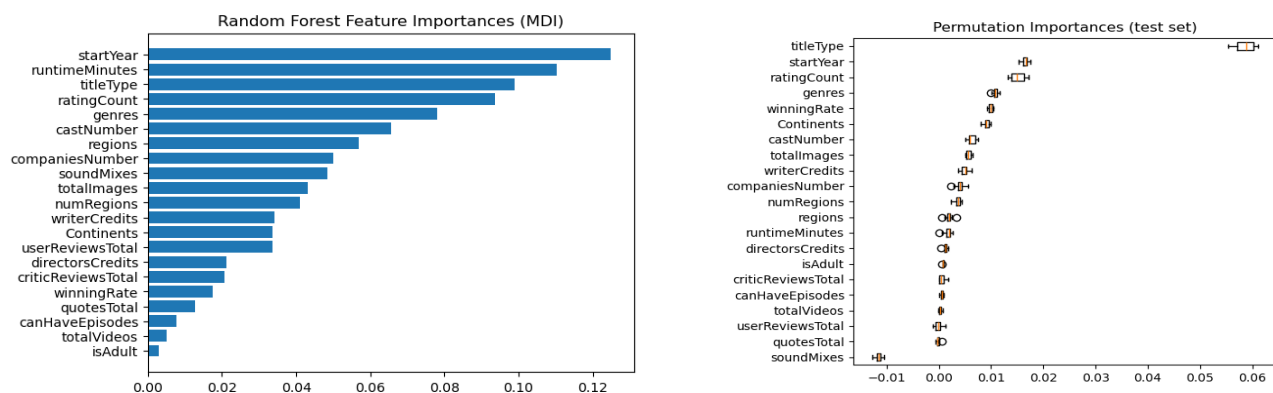


Figure 4.10: Feature Importances and Permutation Importance result

The MDI indicates that *startYear*, *runtimeMinutes*, *titleType*, and *ratingCount*, *genres* are the most important features. These variables were most frequently used by the Random Forest model during training to make splits. Conversely, features like *isAdult*, *totalVideos*, and *canHaveEpisodes* have minimal importance.

Permutation importance aligned with MDI on the relevance of *titleType*, *startYear*, and *ratingCount* and *genres* for the model's predictive ability. However, it ranks *titleType* as the most impactful feature, whereas *startYear* was most dominant during training. *runtimeMinutes*, despite its high MDI importance, displays much lower permutation importance, indicating that while it was helpful for splitting, its predictive power on unseen data is limited. Interestingly, *winningRate* also emerged as influential in the test set, despite being less important in the training set.

4.4.3. Gradient Boosting Classifier

We performed hyperparameter optimization for the GradientBoostingClassifier using RandomizedSearchCV with 5-fold stratified cross-validation. The following parameters were tuned:

'n_estimators': [50, 100, 200, 300, 400, 500],

'learning_rate': [0.001, 0.01, 0.05, 0.1],

'max_depth': [3, 4, 5]

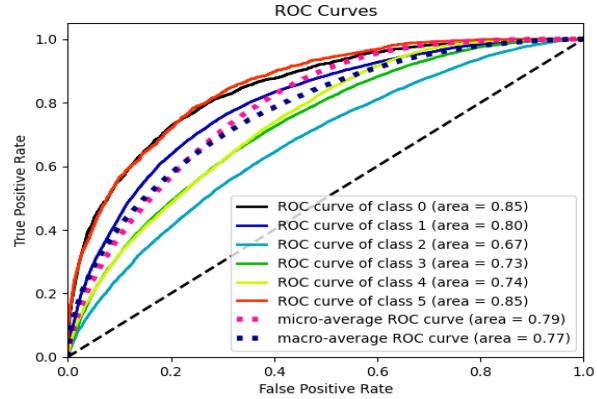
The best configuration selected was : {'n_estimators': 500, 'max_depth': 5, 'learning_rate': 0.1}

Using the best model, the Gradient Boosting classifier achieved the following performance:

Table 4.9: Gradient Boosting result

Class	Precision	Recall	F1-score
0	0.18	0.49	0.26
1	0.49	0.46	0.48
2	0.44	0.28	0.34
3	0.55	0.40	0.47
4	0.31	0.41	0.35
5	0.14	0.53	0.22
Accuracy			0.39
Macro avg	0.35	0.43	0.35
Weighted avg	0.45	0.39	0.40

Figure 4.11: Gradient Boosting ROC Curves



The Gradient Boosting classifier achieved 39% accuracy, a macro-average F1-score of 0.35, and a macro-average AUC of 0.77 performing slightly better than Random Forest. It performed consistently on Classe 1 and Class 3. On tricky classes that are harder to distinguish, Class 2 and Class 4 showed clear improvements (F1 = 0.34 and 0.35; AUC = 0.67 and 0.74). Minority Classes 0 and 5 also benefited, with Class 0 achieving an F1-score of 0.26 (AUC = 0.85), and Class 5 improving to an F1 of 0.22 with recall 0.53. ROC curves indicate strong separability for Classes 0 and 5 (AUC = 0.85). Overall, Gradient Boosting demonstrated improved class discrimination and generalization, particularly for minority and overlapping classes.

4.4.4. Extreme Gradient Boosting Classifier (XGBOOST)

For the XGBoost classifier, we tuned the following parameters using RandomizedSearchCV with 5-fold stratified cross-validation:

'n_estimators': [100, 300, 500],

'max_depth': [3, 4, 5, 6],

'learning_rate': [0.01, 0.05, 0.1],

'gamma': [0, 0.1, 0.3, 1, 5],

'reg_alpha': [0, 0.1, 0.5, 1, 5],

'reg_lambda': [0.01, 0.1, 1, 5, 10]

The best parameter configuration found was:

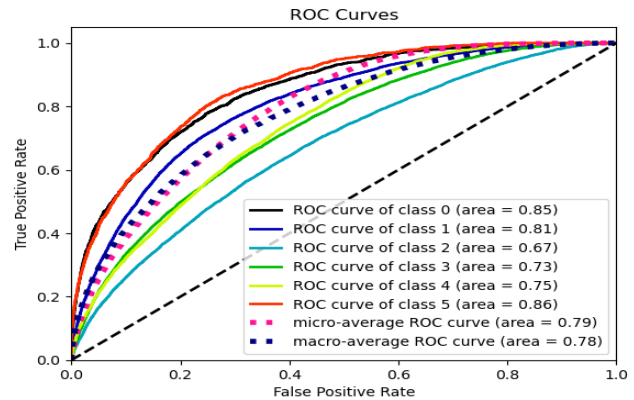
{'reg_lambda': 0, 'reg_alpha': 1, 'n_estimators': 500, 'max_depth': 6, 'learning_rate': 0.1, 'gamma': 0.5}

The optimized XGBoost model achieved the following performance:

Table 4.10: XGBoost result

Class	Precision	Recall	F1-score
0	0.18	0.48	0.26
1	0.49	0.47	0.48
2	0.45	0.28	0.35
3	0.56	0.41	0.48
4	0.31	0.40	0.35
5	0.13	0.55	0.21
Accuracy			0.39
Macro avg	0.35	0.43	0.35
Weighted avg	0.46	0.39	0.41

Figure 4.12: XGBoost ROC Curves



The XGBoost classifier achieved 39.4% accuracy, a macro-average F1-score of 0.35, and a macro-average AUC of 0.78, performing similarly to Gradient Boosting but offering slightly better ROC-AUC. It showed strong results on Classes 1 and 3 (F1 = 0.48) and handled difficult classes like Class 2 and Class 4 effectively (F1 = 0.35 for both). It also demonstrated good class separability for Classes 0 and 5 (AUC = 0.85 and 0.86), although both suffered from low precision. Overall, Gradient Boosting and XGBoost proved effective in managing class imbalance and capturing complex patterns.

Studying the effects of numbers of base model in ensemble methods

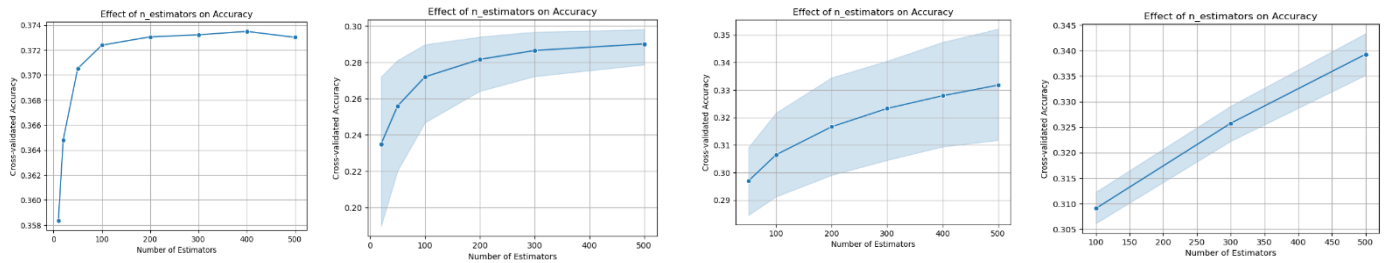


Figure 4.13: Effect of $n_{\text{estimators}}$ on Accuracy for Random Forest, AdaBoost, Gradient Boosting, XGBoost

The number of base estimators significantly impacts classification performance across ensemble models. For **Random Forest**, performance improves rapidly up to around 100 trees, after which gains plateau, suggesting that adding more trees yields minimal benefit beyond this point.

AdaBoost, although also a boosting method that corrects errors from previous learners, primarily relies on decision stumps as base models. As a result, it shows early performance gains, but the improvement flattens between 200 and 500 estimators, indicating limited capacity to capture complex patterns with additional learners.

Gradient Boosting and **XGBoost** show consistent improvements in cross-validated accuracy as the number of estimators increases, confirming that they benefit from deeper model refinement. However, XGBoost exhibits a more linear and stable increase in performance with a tighter confidence interval, indicating greater robustness and lower variance.

In conclusion, for fast and efficient training, the default setting of $n_{\text{estimators}} = 100$ in scikit-learn offers a practical and effective balance for Random Forest and AdaBoost, providing strong performance without fine tuning number of base models. For Gradient Boosting and XGBoost, however, using a higher number of estimators can lead to continued performance gains. For **Gradient Boosting** and **XGBoost**, however, using a higher number of estimators can lead to continued performance gains and therefore require more careful fine-tuning to optimize model performance.

4.5. Final Discussion

To compare the performance of the eight classifiers analyzed above, we used macro-averaged F1-score, ROC AUC, and accuracy as evaluation metrics. The table below summarizes the results:

Table 4.11: Final comparison

Classifiers	Macro-avg F1-score	Macro-avg ROC	Accuracy
Logistics Regression	0.25	0.68	0.35
Linear SVM	0.24	0.68	0.35
Non-Linear SVM	0.24	0.71	0.28
Neural Network	0.32	0.76	0.40
Random Forest	0.33	0.76	0.37
ADABOOST	0.27	0.67	0.30
Gradient Boosting	0.35	0.77	0.39
XGBOOST	0.35	0.78	0.39

- **Logistic Regression** and **Linear SVM** achieving moderate AUC (0.68) but low F1-score (0.25), highlighting their limitations in capturing complex class boundaries and handling imbalanced multiclass distributions.
- **Non-linear SVM** marginally improved AUC (0.71), but its overall performance (F1 = 0.24) did not significantly outperform linear SVM. Considering its higher computational cost and more complex hyperparameter tuning, we preferred Linear SVM over non-linear SVM for this task.
- **ADABOOST** performed worse than other ensemble methods (F1 = 0.27, AUC = 0.67), largely due to its reliance on decision stumps as weak learners, these shallow trees fail to capture intricate patterns in multiclass.
- **Neural Networks** achieved the highest accuracy (0.40) and strong AUC (0.76) but underperformed on minority classes (F1 = 0.32), revealing poor class balance.
- **Random Forest** stood out for fast training and strong performance (F1 = 0.33, AUC = 0.76, Accuracy = 0.37).
- **Gradient Boosting** and **XGBoost** delivered the highest performance (F1 = 0.35, AUC = 0.77–0.78, Accuracy = 0.39), showing excellent capability in managing class imbalance and modeling subtle patterns through iterative refinement.

In conclusion, given the overall complexity of the classification task characterized by the overlapping and imbalanced nature of the six-class rating variable, **ensemble methods** are clearly more effective. We selected **Random Forest** as a fast and reliable model for this task, while **Gradient Boosting** and **XGBoost** are better suited for achieving optimal performance through fine-tuning and handling class imbalance more effectively.

5. Advanced Regression

In this task, we apply several models to predict the *averageRating* variable. The methodologies employed include Neural Networks, Random Forest, and Gradient Boosting.

5.1. Neural Networks

We used Keras Tuner combined with random search for hyperparameter optimization conducted across three distinct model architectures: those with one, two, and three hidden layers. The only difference in comparison with the tuning step at Classification is that we tried to tune the number of each layer.

Once the optimal hyperparameter configuration was identified, the model was trained on the dataset. At that time, early stopping with patience of 20 epochs was also employed.

The specific hyperparameters for this optimal model are detailed in Table 5.1 and the detailed result is presented.

Table 5.1: Best hyperparameter combination and model result

Hidden layers	[96, 16, 16]
Units_1	96
Activation_1	<i>tanh</i>
Dropout_1	0.1
Units_2	16
Activation_2	<i>tanh</i>
Dropout_2	0.1
Units_3	16
Activation_3	<i>tanh</i>
Dropout_3	0.1
Learning rate	0.0001

MSE	MAE	R2 Score
1.2885	0.8427	0.2908

Furthermore, we present the learning curves for the final model in Fig. 5.1 The plots for both training and validation demonstrate that the model is well-fitted and does not suffer from overfitting.

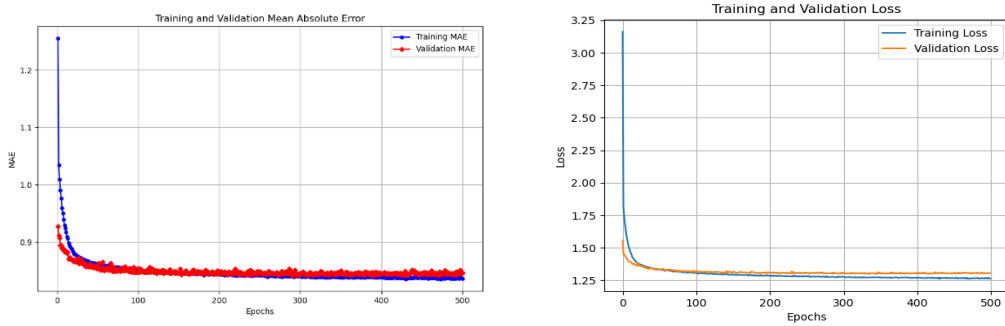


Figure 5.1: Learning curves of the neural network

5.2. Random Forrests

We initiated hyperparameter tuning using a broad Randomized Search. This process identified an initial optimal set: `{'n_estimators': 200, 'min_samples_split': 2, 'min_samples_leaf': 2, 'max_features': 'log2', 'max_depth': 30}`. However, a model with these parameters was severely overfit, evidenced by the large discrepancy between the training set R2 of 0.76 and the test set R2 of only 0.41. This significant discrepancy indicated that the model had learned the noise and specific patterns of the training data rather than its underlying trends, thereby failing to generalize.

We attributed this poor generalization to excessive model complexity from the `max_depth` of 30. To resolve this, we performed a second, focused Randomized Search, constraining `max_depth` to a more conservative range of [9, 12]. This refined tuning yielded a new optimal configuration: `{'n_estimators': 200, 'min_samples_split': 5, 'min_samples_leaf': 2, 'max_features': 'sqrt', 'max_depth': 12}`.

The final model using these parameters successfully resolved the overfitting, resulting in a robust and significantly more generalizable model. The detailed result is shown in Table 5.2.

Table 5.2: MSE, MAE, and R2 for Random Forest

MSE	MAE	R2 Score
1.2557	0.8293	0.3089

We also plot a Feature Importance graph showing that the top 5 important features are *titleType*, *runtimeMinutes*, *startYear*, *genres*, *soundMixes*. This result shows a slight difference in comparison with the Feature Importance graph in Random Forest at Classification step. Besides, the top 5 for both Feature Importance and Permutation Importance for Random Forest Regressor almost have the same result for top 5 most important features except for the difference between '*soundMixes*' and '*ratingCount*'. The key distinction in Permutation Importance is *ratingCount* for the regressor versus *winningRate* for the classifier.

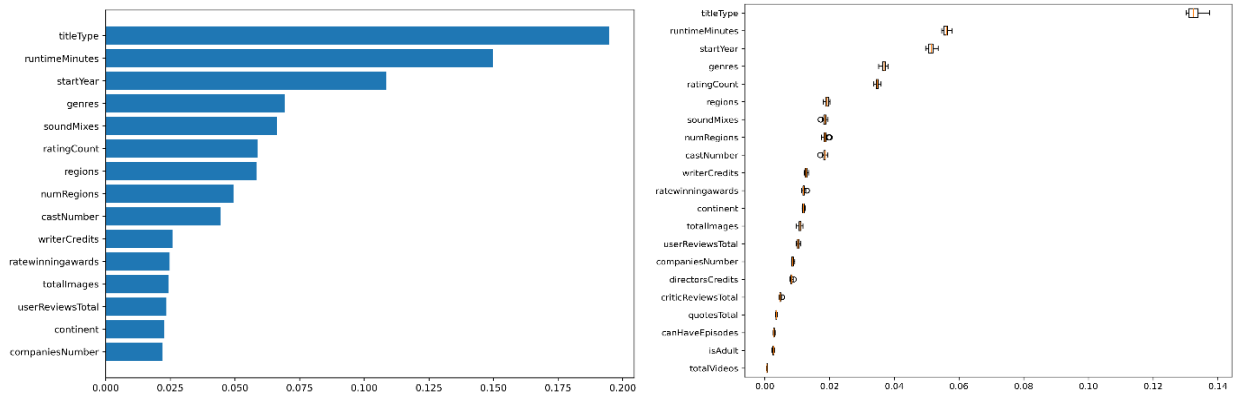


Figure 5.2: Feature Importance and Permutation Importance for Random Forest Regressor

5.3. Gradient Boosting

For this method, we performed hyperparameter tuning, focusing on key parameters that control the trade-off between model complexity and overfitting. The search explored the following:

- *n_estimators* (50 - 400): The total number of boosting trees.
- *learning_rate* (0.001 - 0.2): The contribution of each tree to the final prediction.
- *max_depth* (3 - 5): The maximum depth of each individual tree.

The tuning process identified the optimal parameters as *n_estimators* = 300, *learning_rate* = 0.2, and *max_depth* = 5. With these settings, the model achieved an R² Score of 0.3669 on the test set. This result, detailed further in Table 5.3, highlights the delicate balance required in hyperparameter tuning to create a robust and reliable predictive model.

Table 5.3: MSE, MAE, R2 Score for Gradient Boosting

MSE	MAE	R2 Score
1.1503	0.7858	0.3669

5.4. Conclusion

Overall, all three models exhibit low R^2 scores. This is attributed to the dataset's characteristics, specifically a high level of noise and relationships between variables that are too complex for even advanced models to predict accurately.

Based on the R^2 score, MAE, and MSE metrics, **Gradient Boosting** is the best-performing method. We explain this by its operational mechanism; the method learns sequentially by building each new tree to correct the errors of all preceding ones to achieve the best result. Additionally, Gradient Boosting directly optimizes a loss function to minimize prediction errors.

6. Explainability

In this chapter, we utilized SHAP (SHapley Additive exPlanations) to interpret the predictions made by our XGBoost classifier from the Advanced Classification task. SHAP values tell us how individual features contribute to prediction by comparing them to the average prediction.

We analyzed two individual predictions that were both classified into Class 5, one correctly classified instance and one misclassified instance.

The waterfall plot Fig.6.1 illustrates the SHAP explanation for instance 786, which was correctly predicted as Class 5 with:

- $f(x) = 2.429$ is the predicted log-odd of instance 786
- $E[f(x)] = 0.405$ is the average predicted log-odd of class 5 across all instances.

For this observation, the feature *startYear* = 2023 had the highest positive SHAP value (+0.73) significantly increased the prediction toward Class 5. This was followed by *genres* = 68909 and *runtimeMinutes* = 30, which contributed +0.34 and +0.28, respectively. Additional features such as *soundMixes* = 78773, *companiesNumber* = 0, and *titleType* = 70824 also added positively to the prediction, although with slightly smaller effects. Only a few features, such as *Continents* = 23845, had a slight negative effect.

For the second observation instance 2244 (see Waterfall plot Fig. 6.1), was misclassified as Class 5, even though its true class was Class 4. In this case, the model predicted a log-odds of 1.324 compared to the average of 0.405. The top positive contributors were *ratingCount* = 9 (+0.41), *startYear* = 2014 (+0.25), and *soundMixes* = 78773 (+0.22). These features pushed the model toward Class 5. However, a key difference was the feature *criticReviewsTotal* = 1, which had a substantial negative contribution of -0.37. Despite this negative signal, the overall sum of positive contributions resulted in the model assigning this instance to Class 5.

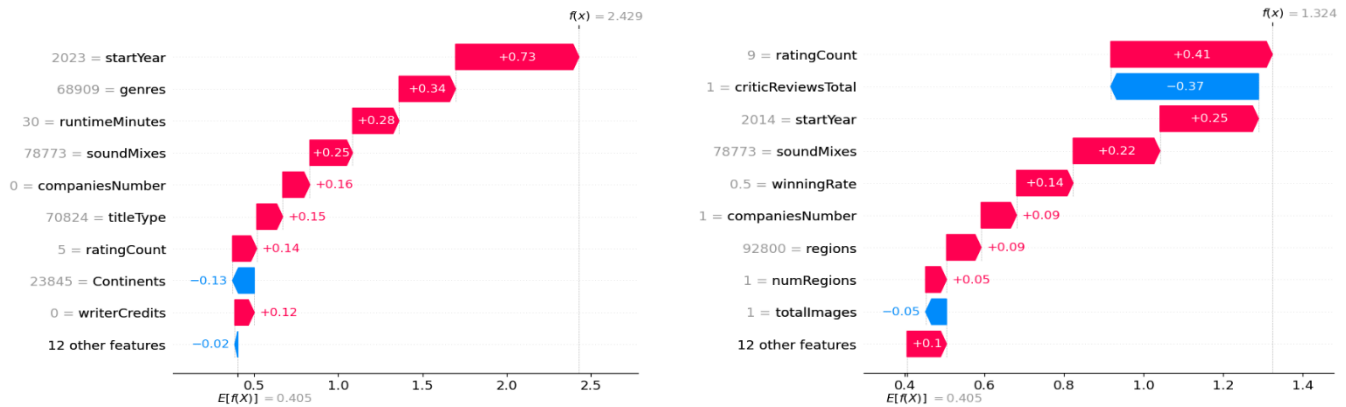


Figure 6.1: SHAP prediction for instance 786 (left) and 2244(right)

To better understand how the model works, particularly its decision-making behavior for Class 5, we also incorporated the mean SHAP value plot (Fig. 6.2) and the SHAP beeswarm plot of Class 5 (Fig. 6.3). The mean SHAP plot revealed that features such as *startYear*, *ratingCount*, and *runtimeMinutes* consistently had the highest impact on Class 5 predictions. In contrast, Class 4 shows lower SHAP impact across all features, with the most contributing features being *titleType*, followed by *ratingCount* and *startYear*, *runtimeMinutes*. The SHAP beeswarm plot for Class 5 further confirms these insights, showing that high values of *startYear* and *ratingCount*, low values of *runtimeMinutes* strongly push predictions toward Class 5. This suggests that the model tends to associate recent, short-duration, and highly rated titles with Class 5.

We tried to debug the model’s misclassification of instance 2244 by comparing its feature values against the global SHAP patterns. Although it had a strong negative signal from *criticReviewsTotal*, the presence of high *ratingCount*, a relatively recent *startYear*, and contributing *soundMixes* values matched the typical Class 5 profile. This reveals that the model heavily favors dominant features such as *ratingCount*, *startYear*, and *soundMixes* in its decision-making of instance 2244 leading to misclassification.

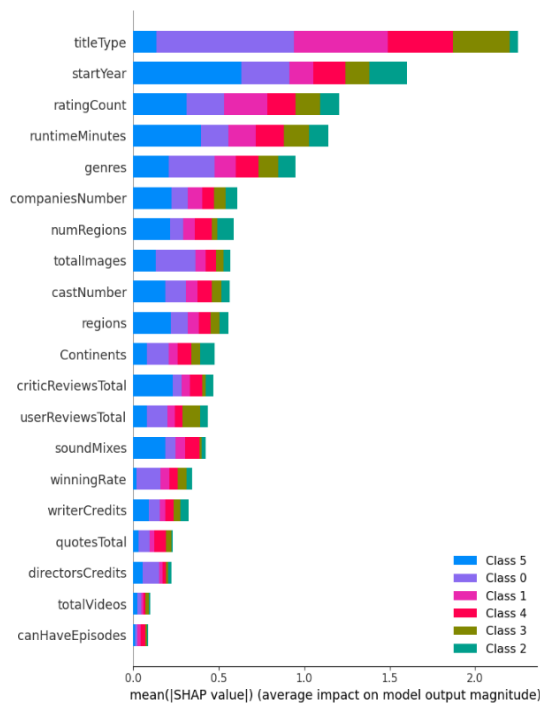


Figure 6.2: Mean SHAP value

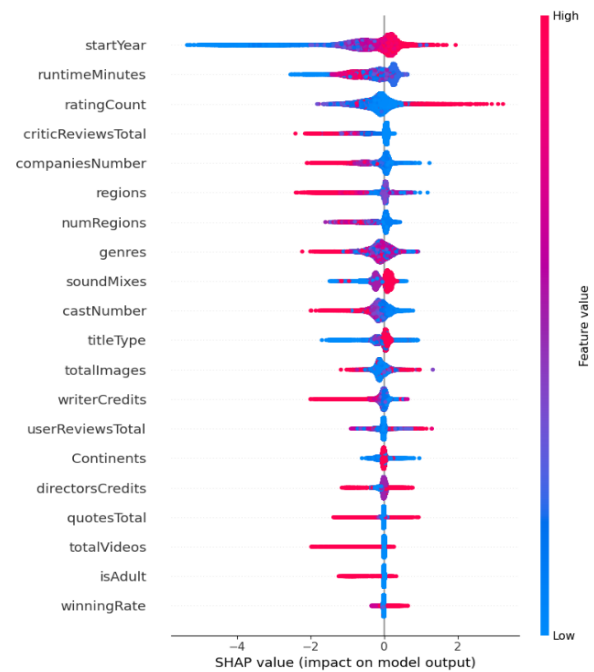


Figure 6.3: Bee swarm Plot of Class 5

7. Time Series

We analyze 1,134 films with daily domestic box office (US & Canada) for the first 100 days after theatrical release, sourced from IMDbPro. Fields include movie *ID*, *rating*, *rating category*, and *genre*.

7.1. Understanding and Preparation

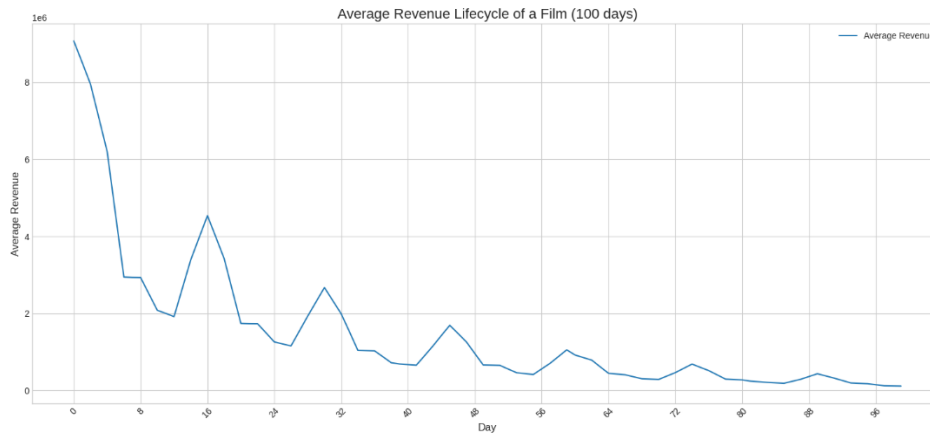


Figure 7.1: Average Revenue Lifecycle

Revenue lifecycle: Revenue peaks on the opening weekend (avg $\approx 9,075,217$; most openings on Friday), then drops $\sim 50\text{--}60\%$ within a few days. Clear weekend seasonality, with larger peaks roughly every two weeks—likely tied to marketing pushes. Weekdays (Mon–Thu) are troughs. After ~ 40 days, revenue is low but more stable; weekend effects persist but weaken.

By genre: Most genres follow this curve. **War** and **Western** underperform from the start. **Western** shows a late surge around day ~ 75 , likely due to a Western-related event that boosted interest.

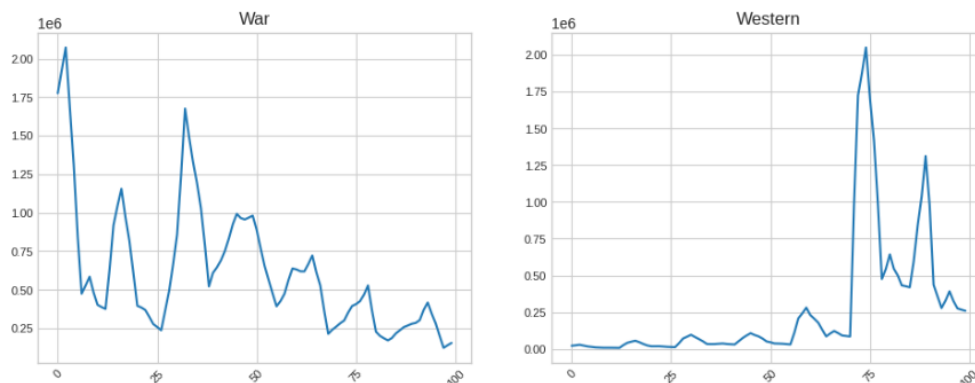


Figure 7.2: Average Revenue for War and Western

Ratings relationships: Higher ratings correspond to higher average revenue (ordered High $\rightarrow$ Low). By genre, ratings are highest for Documentary, Short, War and lowest for Family, Mystery, Horror. By revenue, top genres are Sci-Fi, Adventure, Animation; lowest are Documentary, Western, Short.

After quality checks, no missing values were found. We screened anomalies using a cross-validated ensemble (Hampel filter, Grubbs' test, IQR), applied both globally and with sliding windows; any point flagged by $\geq 3/5$ detectors would be removed, but none met this criterion. We then standardized all series with Z-score normalization to support fair clustering and classification. Given the dataset's moderate size, we kept the original

series rather than applying complexity-reduction (e.g., PPA, SAX). For classification only, we merged the rare **Low** ratingCategory (<1%, n=10) into **Medium Low** to improve class balance.

7.2. Clustering

In this phase, a distance-based clustering approach is applied using K-means, as well as DBSCAN clustering after transforming the dataset into new features using some methods to generate new features.

7.2.1. K-means

We applied TimeSeriesKMeans with both Euclidean and DTW distances. For each distance, we chose k using the Elbow, Silhouette, Calinski–Harabasz, and Davies–Bouldin criteria. The best k was 5 for DTW and 4 for Euclidean. We proceeded with the Euclidean solution because t-SNE showed clearly separated clusters with minimal overlap, unlike DTW. The “Cluster Centroids with STD” plot shows each cluster’s mean trajectory and its standard deviation, summarizing typical patterns and variability.

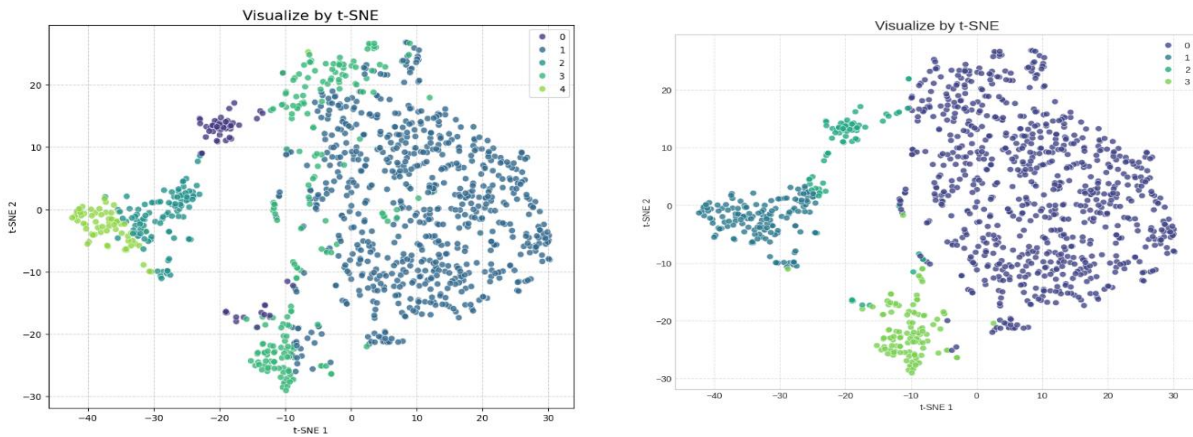
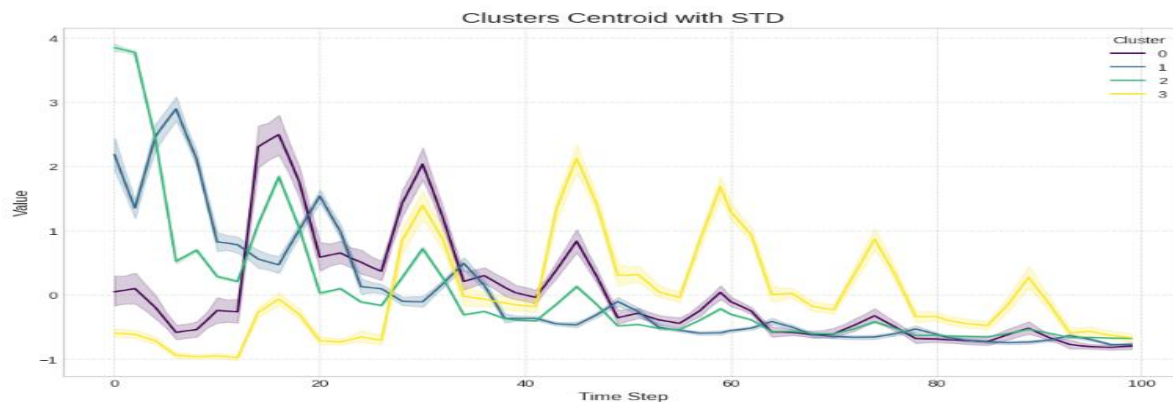


Figure 7.3: Visualize by t-SNE for Clusters

Cluster 0: starts low, with many ups and downs and sharp peaks (around days 18, 30, 48). The spread is wide, so sales vary a lot. **Cluster 1:** low and steady. The spread is very narrow, so series in this cluster are very similar with little change. **Cluster 2:** very high at the start, then drop quickly. This looks like an early “boost” or big event. **Cluster 3:** big swings with tall, sharp, repeating peaks. These peaks are much higher than other clusters. This cluster changes the most.



7.4: Clusters Centroid for K-means

Figure

By rating_category, Cluster 2 is the largest and the only cluster containing all five rating levels. The other three clusters are mostly **High**, **Medium High**, and **Medium**.

The PCA plot shows clear separation, consistent with t-SNE. Cluster 0 forms a distinct arc. Clusters 1 and 3 also appear as separate groups. Cluster 2 is more spread out but still separated from the others.

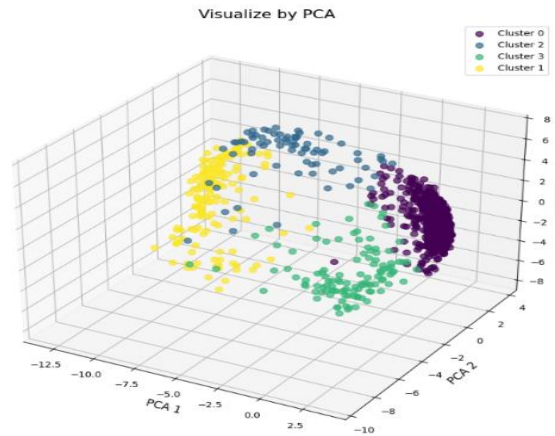


Figure 7.5: Visualize by PCA for K-means

7.2.2. DBSCAN

We built features with three methods—Custom Feature Extraction, Summary Transformer, and TS-Fresh—creating separate datasets. For each, we tuned DBSCAN using KneedleLocator to choose eps and min_samples. The best setup used TS-Fresh features with eps = 1.49e17 and min_samples = 5, giving the highest Silhouette (0.897) and almost no noise (only one point).

However, PCA and t-SNE show DBSCAN grouped almost everything into one large cluster (Cluster 0), with only a few noise points (-1) and two tiny clusters (1, 2). In short, DBSCAN did not find meaningful groups. The likely reason is the need for a very large eps, which suggests the data lack clear density separation in this feature space.

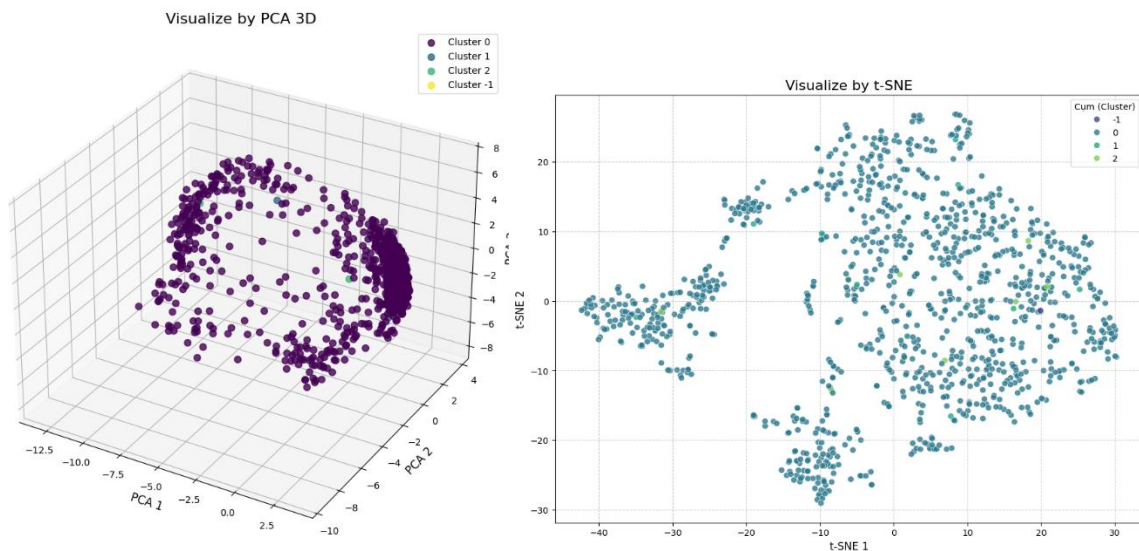


Figure 7.6: Visualize by PCA and t-SNE for DBSCAN

7.3. Classification

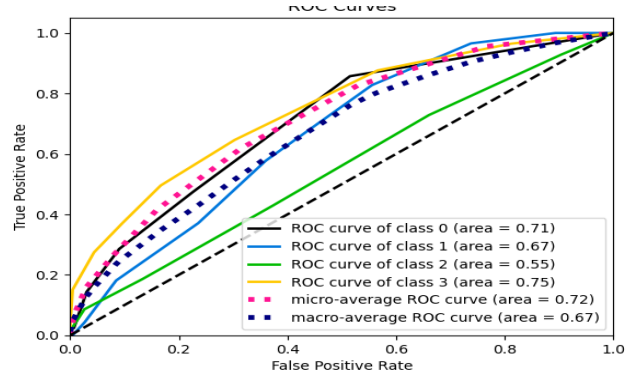
7.3.1. KNN with Euclidean distance

We tuned KNeighborsTimeSeriesClassifier using GridSearchCV with 5-fold stratified cross-validation to find the optimal number of neighbors in the range [3, 5, 7, 9, 11]. The best neighbor found was 9. KNN achieved the following performance:

Table 7.1: KNN with Euclidean distance result

Class	Precision	Recall	F1-score
Low	0.35	0.21	0.26
Medium	0.45	0.69	0.54
Medium High	0.28	0.21	0.24
High	0.65	0.47	0.54
Accuracy			0.46
Macro avg	0.43	0.40	0.40
Weighted avg	0.65	0.47	0.54

Figure 7.7: KNN with Euclidean distance ROC Curves



Per-class results showed stronger performance on High and Medium (F1 = 0.54), while Low (F1 = 0.26) and Medium High (F1 = 0.24) remained challenging. ROC analysis indicated Class Low (AUC = 0.71) and Class High (AUC = 0.75) were best separated, with Class Medium High the hardest (AUC = 0.55).

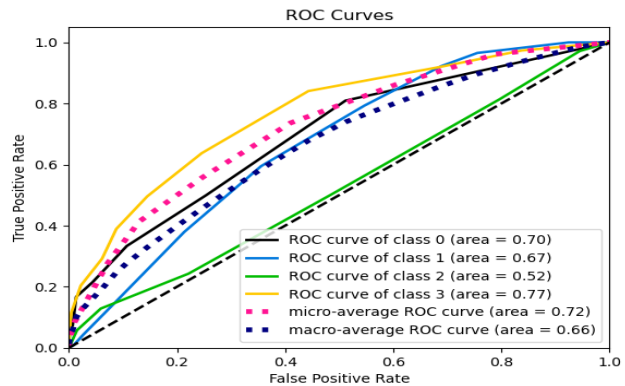
7.3.2. KNN with DTW distance

We used same method as above but with Dynamic Time Warping as distance metric incorporating a Sakoe-Chiba global constraint of 7 days. The best neighbor found was 11. Using this setting, KNN with DTW achieved:

Table 7.2: KNN with DTW distance result

Class	Precision	Recall	F1-score
Low	0.39	0.17	0.23
Medium	0.46	0.78	0.58
Medium High	0.20	0.21	0.15
High	0.62	0.49	0.55
Accuracy			0.47
Macro avg	0.42	0.39	0.38
Weighted avg	0.45	0.47	0.44

Table 7.8: KNN with DTW distance ROC Curves



KNN with DTW distance, achieved a small gain in overall accuracy compared to Euclidean distance. The most notable improvement was for the Medium class, where recall increased to 0.78 (from 0.69 with Euclidean), making DTW much better at identifying this majority class. The High class also showed solid performance (F1 = 0.55, AUC = 0.77). This shows DTW's ability to handle temporal distortions and align frequent patterns more effectively. However, DTW was weaker in underrepresented classes such as Low (F1 = 0.23) and Medium High (F1 = 0.15, AUC = 0.52), where Euclidean distance performed slightly better.

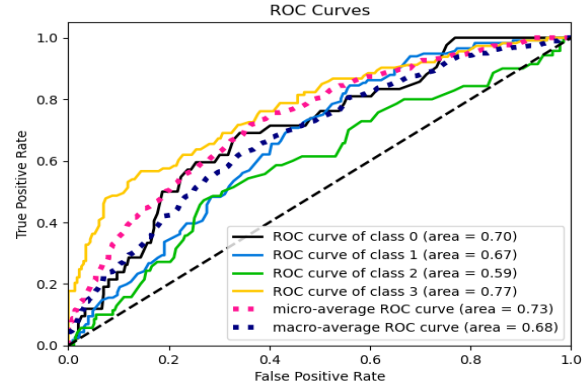
7.3.3. Shapelets

We applied the Random Shapelet Transform to extract discriminative subsequences from the time series before classification. The transform was configured to sample 10,000 candidate shapelets, retaining the top 300 most informative shapelets based on their predictive power. The minimum and maximum shapelet lengths were set to 7 and 21, respectively, with self-similar shapelets removed to reduce redundancy. A Random Forest classifier was then trained on the transformed feature space.

Table 7.3: Shapelets result

Class	Precision	Recall	F1-score
Low	0.17	0.02	0.04
Medium	0.43	0.75	0.55
Medium High	0.40	0.14	0.21
High	0.61	0.59	0.60
Accuracy			0.48
Macro avg	0.40	0.38	0.35
Weighted avg	0.45	0.48	0.44

Figure 7.9: Shapelets ROC Curves



The Random Shapelet Transform with Random Forest achieved a slightly higher accuracy of 48% and a Macro AUC of 0.68, but a lower Macro F1-score of 0.35 compared to the two precedent models. Similar to KNN with DTW distance, shapelets showed strong performance for the High class (F1 = 0.60, AUC = 0.77) and the Medium class (F1 = 0.55, AUC = 0.67), demonstrating their ability to capture discriminative local subsequences for these categories. However, the Low class (F1 = 0.04, AUC = 0.70) and Medium High class (F1 = 0.21, AUC = 0.59) remained challenging, with Low suffering from particularly poor recall (0.02), despite showing reasonable separability in the ROC analysis.

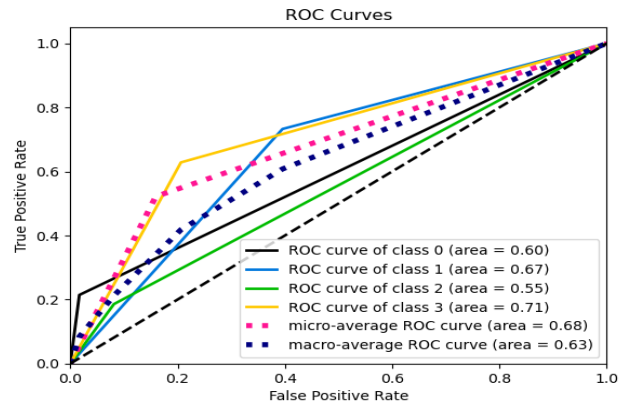
7.3.4. Rocket

We trained a RocketClassifier using the MiniRocket transform and tuned hyperparameters with GridSearchCV under 5-fold stratified cross-validation. The search explored the number of kernels [5000,10000,20000] and the maximum dilations per kernel [8,16,32, 64]. The best configuration selected was `num_kernels=5000`, `max_dilations_per_kernel=64`, `rocket_transform='minirocket'`.

Table 7.4: Rocket result

Class	Precision	Recall	F1-score
Low	0.64	0.21	0.32
Medium	0.49	0.73	0.59
Medium High	0.37	0.19	0.25
High	0.60	0.63	0.61
Accuracy			0.52
Macro avg	0.53	0.44	0.44
Weighted avg	0.52	0.52	0.49

Figure 7.10: Rocket ROC Curves



Performance was strongest on the High ($F1 = 0.61$, $AUC = 0.71$) and Medium classes ($F1 = 0.59$, $AUC = 0.67$), indicating that RocketClassifier effectively captured discriminative convolutional features for the majority categories. The Low class achieved an F1-score of 0.32 ($AUC = 0.60$), while Medium High remained challenging with an F1-score of 0.25 ($AUC = 0.55$). Compared to Euclidean, DTW-based KNN and Shaplets, RocketClassifier provided highest overall accuracy and strongest balance between precision and recall with highest $F1 = 0.44$. Although its separability for minority classes was moderate, RocketClassifier offered a clear improvement in generalization and efficiency, making it the best-performing method among the tested approaches.

7.4. Motifs

In this task, we identified the top 3 motifs in each movie's box office time series to uncover common revenue patterns. Using a subsequence length of 7 days to reflect weekly trends, we computed the matrix profile with the stumpy library to find the most similar subsequence pairs (top motif pairs). Each motif set was then expanded by including all subsequences within a radius of 1.25 and an exclusion zone was applied to avoid trivial overlaps.

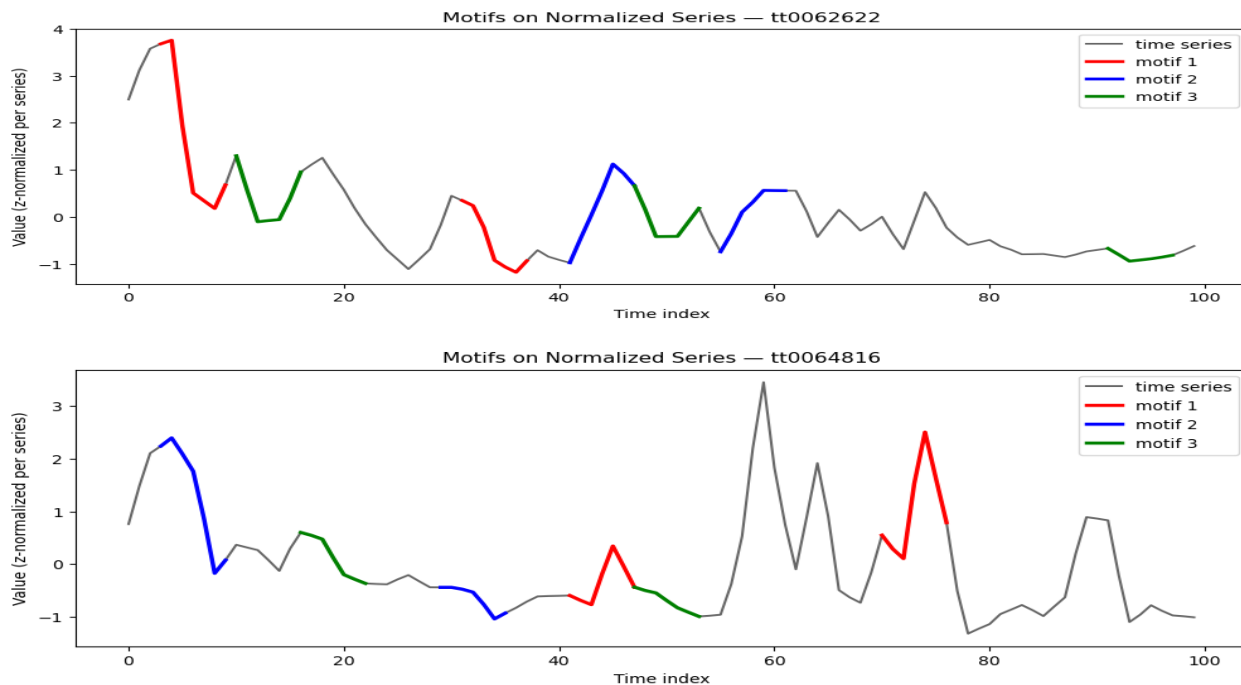


Figure 7.11: Motifs Visualize

We tried to interpret the motifs to better understand patterns in movie revenue dynamics.

For movie tt0062622, Motif 1 appears at the very beginning of the run captures the sharp surge and steep drop that corresponds to the opening weekend and immediate decline. This pattern is repeated at days 3 and 31, showing that a similar weekly shape occurred both in the first week and again later in the first month. Motif 2 is located in the mid-phase at day 41 and 55 capture moderate surge in revenue. Motif 3 has patterns repeated at days 10, 47, and 91 across the early, mid, and late phases of the run. This indicates a consistent repetition of revenue fluctuations.

For movie tt0064816, Motif 1 appears at days 41 and 70, capturing a sharp upward surge followed by a steep decline. This pattern occurs in the mid-to-late phase of the run and likely reflects a sudden burst of popularity. Motif 2 is found in the earlier part of the series, specifically on days 3 and 29. It is a common pattern during the opening weeks with an initial rise and steep decline, Motif 3, found at days 16 and 47, shows a declining trend after a small surge indicating gradual loss in audience attention.

Furthermore, the distribution of motif start indices (Fig. 7.12) reveals clear patterns in how motifs are positioned throughout the 100-day time series. We observed that motifs are not evenly spread across the 100 days but tend to cluster at specific time points. Plot showed that clear peaks appear around days 2, 10, 16, 24, 30, 39, 45, 53, and 74, among others. These positions occur at roughly weekly intervals, reflecting the strong influence of 7-day revenue cycles.

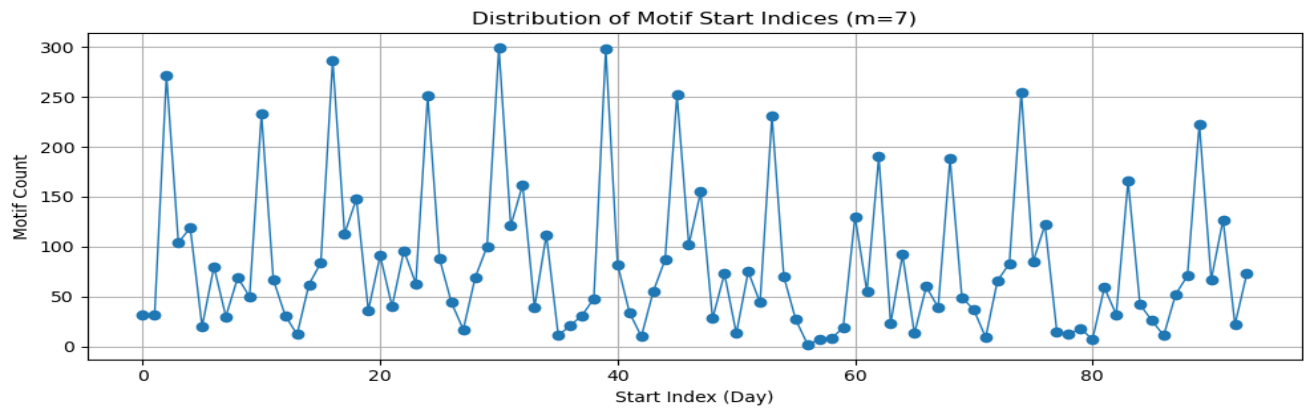


Figure 7.12: Distribution of Motifs

7.5. Compare with Shapelets

Motifs capture common revenue behaviors across the same movie's timeline. While Shapelets capture discriminative subsequences that separate one rating category from another.

In our analysis, motifs were extracted with a fixed length of 7 days to reflect weekly audience cycles. Shapelets, in contrast, varied in length from 7 to 20 days. Shorter shapelets (7–10 days) often overlap with motifs, capturing opening-weekend surges or weekly oscillations. Longer shapelets (15–20 days) extend beyond motifs, embedding one or more motifs within a broader temporal context, such as a sustained decline over two or three weeks.

Another key difference lies in temporal positioning. Motifs clustered at consistent weekly intervals across all movies, whereas shapelets showed category-specific positioning. Category 1 shapelets were long and concentrated early in the series (days 0–20), capturing opening-weekend patterns. Category 0 shapelets appeared later (days 50–90) and captured delayed bursts of popularity. Category 2 shapelets were short, weak, and scattered in the mid-to-late phase (days 30–70). Category 3 shapelets were long, high-gain subsequences concentrated in the early-to-mid phase (days 0–40), often reflecting oscillatory cycles.

Finally, in terms of discriminative power. Motifs highlight subsequences that are frequent but not necessarily discriminative. Shapelets, by contrast, are ranked by gain, which measures their ability to separate categories. High gain shapelets were often longer subsequences that included motifs but extended them with additional context, such as multi-week oscillation. Category 3 exhibited the highest gain shapelets, Category 1 moderately high, Category 0 moderate, and Category 2 the lowest.

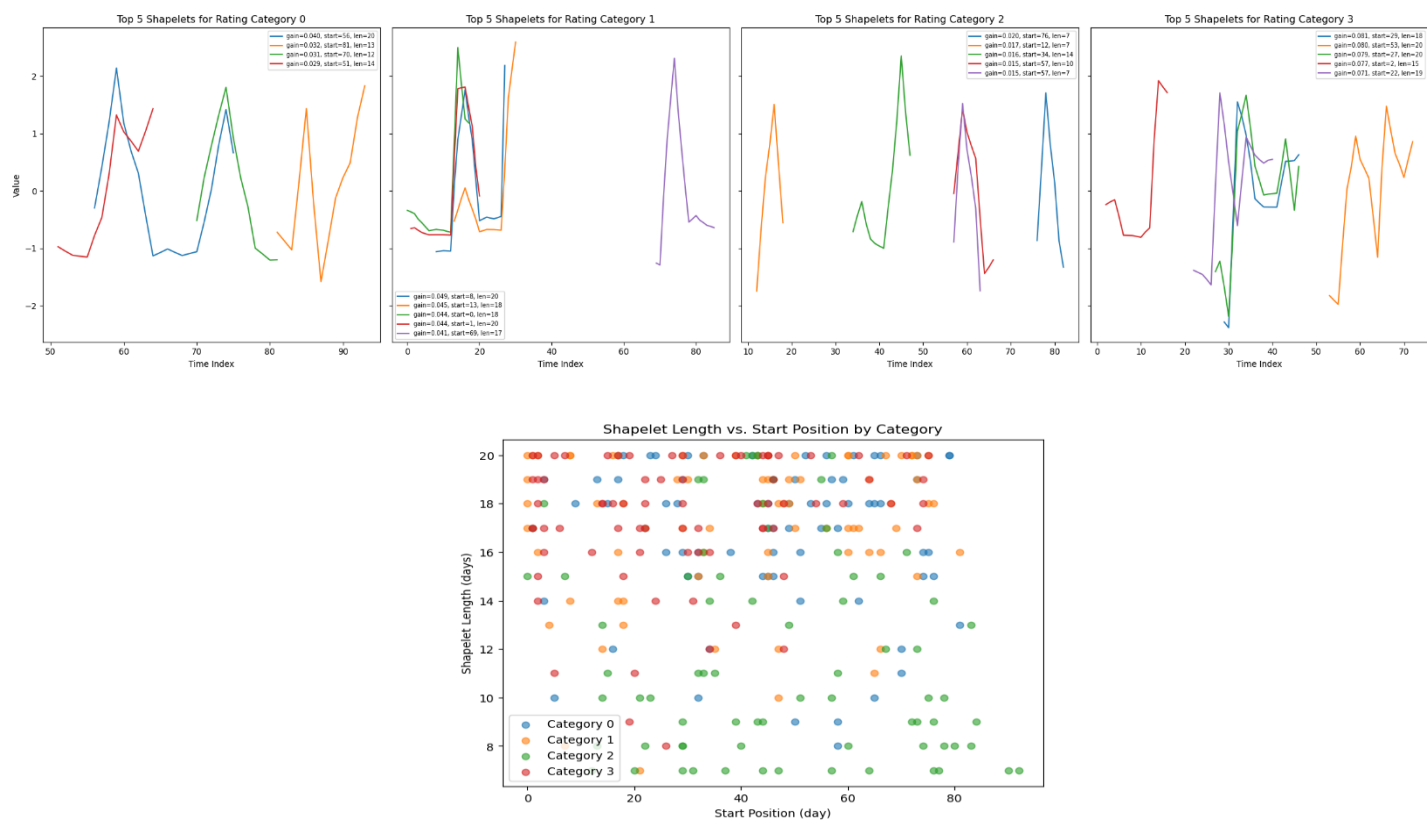


Figure 7.13: Shapelet Visualization